



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECP2523: DATABASE (WBL)

SEM I 2025/2026

PROJECT 2 - CONCEPTUAL DATABASE DESIGN

SYSTEM: HASTA CAR RENTAL SYSTEM

Group Name: SpinTheWheel

Members:

- 1) **Chun Yao Ting (A24CS0239) [Team Leader]**
- 2) **Evelyn Ang (A24CS0068)**
- 3) **Mirza As-Siddiq Bin Tohari (A24CS0112)**
- 4) **Muhammad Aliff Bin Helmi (A24CS0272)**

Stakeholder: Hasta Travel & Tours Sdn. Bhd.

Representative Name	Date Interviewed
Encik Alif	07/11/2025

Printing Date: 03/02/2026

Table of Contents

1.1	Introduction	3
1.1.1	Overview of Company	4
1.1.2	Problem Statement	6
1.1.3	Proposal	8
1.2	DFD (TO-BE)	20
1.3	Proposed Business Rule	22
1.4	Database System Requirements	25
1.4.1	System Overview	25
1.4.2	Module <Authentication>	27
1.4.3	Module <Booking>	30
1.4.4	Module <Pickup & Return>	33
1.4.5	Module <User>	37
1.5	Global Conceptual Data Model (Global ERD)	41
1.5.1	Global Conceptual Design (ERD)	41
1.5.2	Enhanced ERD (EERD)	42
1.5.3	Verified Global Conceptual Design (Verified ERD)	43
1.5.4	Data Dictionary of Global Conceptual ERD	44
1.6	Summary	61

1.1 Introduction

In this phase, this report describes the database conceptual design for the Hasta Car Rental System. The objective of this phase is to effectively translate the system requirements determined in Phase 1 into an organized and well-structured database design to accommodate the entire rental process, from customer registration and activation to rental booking and payment processing, rental pickup and return documentation, refund processing, voucher rewards, late return penalties, and staff management.

This report includes the definition of business rules, data, and transactional requirements to clearly show how the system data should be maintained and accessed. A DFD (to-be) is provided to represent the information flow from users to the system. A Conceptual ERD and an Enhanced ERD (EERD) are also created to effectively outline the key entities and attributes and the relationship and constraints between them, as determined by the system requirements. Lastly, a data dictionary is provided to demonstrate each entity and attribute to ensure an unambiguous database design.

1.1.1 Overview of Company

Hasta Travel is a company operating in Universiti Teknologi Malaysia (UTM), mainly providing their car rental service for UTM students and staff. Hasta Travel provides a highly customizable service for their customers such as delivery service for out-station pickup and return, booking outside working hours, short period booking within one day, cancel booking before one day of pickup and so on. Due to their small customer base, the company staff are still able to manage the operation of the company that highly relies on the manual processes and human judgment on eligibility verification, office-hour constraint and location charges. Although this manual approach allows flexibility and still can fulfil the current requirements, this operation mode will fall off once the booking volume increases as the company grows over the time. Additionally, Hasta also decided to transition from physical operation to mixed operation, where customers can do their booking offline or online. This means that the current operation mode already cannot address the future digitalized booking mode.

Due to the absence of a centralized database, it is very difficult for staff to maintain the accuracy of data and make sure the data is up to date. For example, booking availability is checked manually, which is difficult to ensure the vehicle booked is not double booked under overlapping booking periods. Besides, payment verification, booking approval and vehicle status updates are also handled independently, which cause delays and inconsistency across records. In fact, the real practice is the data transaction should reflect changes on other related data. Hence, we can observe that there is a critical issue of a lack of real-time data processing, which is disastrous for car rental services that are sensitive to time.

Other than that, the current system also stores customer documents, such as driving licenses, matric cards, identification cards, inspection form, agreement forms and additional documents in different formats and different systems. The storage of data is unorganized and unstructured, and this causes the historical customer and booking records to be very hard to retrieve in future. The lack of proper data integrity constraints also might cause invalid or incomplete data to be recorded. Additionally, staff also need to manually compile booking statistics, revenue summaries, and vehicle usage records, which is inefficient and increases the rate of human errors. Without a relational database, data cannot be easily queried to support decision making.

Overall, we can see there are many limitations in current business operation mode. Therefore, we need to implement a centralized relational database to store customer, vehicle, booking and transaction data in a systematic way, with clearly defined relationships between data and entities. This allows real-time updates in data, as well as reduces data redundancy by normalization, and also ensures data consistency and integrity for all operations. With this proposed solution on database, efficient querying of data and transaction handling can be fulfilled, which can reduce staff workload and overall service quality for customers.

1.1.2 Problem Statement

Based on the case study, we can observe that Hasta Travel still uses a fully manual and fragmented workflow in the company operation, such as the process of booking, pickup, return and customer management. This had caused a heavy workload on the staff side, resulting in operational efficiency, and increasing risk of human error. Meanwhile, for the customer side, the booking, pickup and return process is also inefficient, which affects the customer experience and brings a negative effect on company image.

Currently, the booking process fully relies on WhatsApp communication, which requires staff to repeatedly prompt customers to send information, manually verify their eligibility to book, check booking constraints such as location, working hours and minimum rental period, as well as determine additional charges for different types of special cases. Meanwhile, customers also have to manually send the details, and their details are not stored, meaning that they still need to send the send information again during the next booking, which is unnecessary and complicated for the customer booking process.

Besides, staff even need to check the car availability to approve the booking by using a physical whiteboard and a separate staff system which is Wahdah. They also need to manually record down booking details across different platforms, including WhatsApp, Wahdah, physical whiteboards, notebook, and Excel daily sales report. This is a bad practice for data transactions, as the data redundancy, delays and errors are likely to occur.

Furthermore, staff also must manually generate, print, fill, verify and store inspection and agreement forms signed by customers during pickup. Important information such as mileage, fuel level, pickup time and staff in charge are required to be filled in by staff manually, as these details are also not automatically inserted. The document is also stored in an unorganized way, as some of them will be in digital documents, some of them will be manual, so it is hard to track the legal documents if needed. Both online and physical pickups require human intervention, like document handling and updates to sales records repeatedly.

Additionally, staff also depend on external GPS tracking systems as well as manual inspection to record mileage, fuel level, late return penalties, and damages. Any issues during rental such as delays or damages will require documentation, customer communication and penalty calculation, all by manual work from staff. Other than that, deposit refunds are also tracked manually, and return information also needs to be recorded into Excel. This provides no flexibility to customer and staff, and also contributes to repetitive workload for staff.

Overall, the absence of integrated and automated booking and management system causes excessive manual data entry across different platforms, high dependency on staff accuracy, judgment and availability, long processing time for booking and transactions, high risk for data inconsistency, lost records and human error, poor customer service due to long response time and repeated information request, and no scalability to address future growth in company size and booking volume. As Hasta also wishes to have a more flexible workflow by transiting to off-station booking, this requires a full revolution in the current system.

1.1.3 Proposal

1.1.3.1 Proposed System

To improve the current system workflow, we came out with our proposed solution which digitalizes the car rental system and organizes the data systematically in the database. We proposed the system to have both the customer side and staff side. The customer side is where the customers do the booking, while the staff side is for the staff to access and manage all the bookings. Next, we will explain the overall workflow of our proposed system.

First, as currently customers need to fill the information again and staff need to record the information again during each booking, we propose a user management system where Hasta staff and customer information will be stored in their respective accounts. This will be covered in the authentication module which include register, login, activation, and access control. Staff and customer will use the same login page but will be redirected to staff side or customer side based on their respective role.

To register, the customer has to fill in their full name as per NRIC, email, and their password, and each email should be unique, meaning that email cannot be reused for different accounts. After registration, customers can login into their account by entering correct credentials, but they still cannot do booking because their account is not activated yet. To activate an account, customers have to fill in all the information required such as IC/passport, nationality, UTM ID (Staff/Student), contact, emergency contact, emergency contact relationship, license number, license expiry date, college, college block, faculty, and home address. Afterwards, they also have to submit a front & back picture of IC/passport, UTM matric card, and car license. The submitted information will be sent to the staff side for manual verification. If staff notice any issue, they can decide to reject it with a reason, else they can approve the activation and customers can start booking now. For activation requests that are rejected, customers may resubmit the activation request and wait for staff approval. Additionally, the system will also send email to notify successful or unsuccessful activation.

Now, we propose the digitalized booking process, which is much easier and more convenient than the current booking process. First of all, customers need to select the preferred date, time, and location for both pickup and return. The system will filter based on the time range of booking and display available cars during that period, with the car model, car plate, features and price. Then, customers can select the preferred car and proceed to the payment page. The payment page will have the payment breakdown section to display all the costs such as rental cost, deposit of RM50, and delivery fee (if the pickup location and return location is not in UTM). Customers also could use vouchers to gain discounts and the discounted price will be shown as well. As the stakeholder does not want to implement the online payment gateway, hence payment screenshot is still used for payment submission. Customers can either pay by QR or using Hasta bank details shown in the payment page. Last but not least, they need to submit the receipt or payment screenshot for verification purposes.

Currently, the booking is under pending status. So, staff will review the customer booking and check if the payment receipt is correct or not, and double check the availability of the car. If there is no issue, staff can approve the booking, else they can reject the booking by stating the reasonings and issue the rejection refund by submitting a refund receipt. Customers will also receive email for booking approval and rejection. If customers wish to cancel their booking, they can do so before 24 hours of pickup time. Then, staff need to review it and issue the cancellation refund by submitting a refund receipt.

After booking approval, customers can do pickup before 30 minutes of pickup time, and there will also be email reminders to notify customers to pick up their car. There will be a few steps to finish the pickup process. First, customers have to review the booking details. Secondly, customers need to download the inspection form and agreement form, fill it, and submit the filled form in the system. Thirdly, customers need to take 5 pictures, which are front, back, left, right and odometer of the car and submit it to the system. Finally, customers can finish the pickup by clicking submit, and the rental status will become picked up.

To return the car, customers can issue a return before 24 hours of return time, and customers will receive an email reminder as well. There will be some steps to complete the return as well. First, customers need to review the booking details. Secondly, customers have to submit 6 pictures, which are front, back, left, right, odometer and car key of the car. Thirdly, customers have to submit their feedback, if there is any issue during booking, customers can press the tickbox in the page as well. Fourthly, customers need to fill in their bank details which are cardholder name, bank name, and account number for deposit refund later on. They may use an existing bank detail if they stored the information in their profile. Finally, customers can submit the return form, and the rental status will change to returned.

After the car returns, there are few tasks that need to be done by the staff. First, staff need to check the feedback of customers for this booking. If there is any issue, the car will automatically become maintenance status. Only after the staff solve the issue (e.g. Aircond no function), staff can click “Resolve” and the car will be available. Secondly, staff need to update the mileage and fuel bar of the cars based on the odometer submitted by the customer. Finally, the booking is finally completed after staff issued the deposit refund by submitting the refund receipt.

For late return, the status of booking on the customer side will be changed to late return after 30 minutes of original return time. The penalty imposed will be based on the return time, for every hour after 30 minutes of original return time will be charged for RM50. Customers have to complete the return process first. Afterwards, their account will be directly blacklisted and there will be one pop out to ask the customer to pay the penalty first. Customers cannot do any new booking as their account is suspended. After they submit the payment receipt for penalty, they still need to wait for approval and their account remains suspended. So, the staff will check the payment receipt submitted by the customer and approve it if there is no issue. After that, customers will remove from the blacklist and can do booking as normal.

Now, we will explain our proposed solution to manage vouchers. There will be two methods of gaining vouchers, one generated automatically by system and another one by staff doing voucher assignments to customers. For automatically generated vouchers, it can be generated in two ways, the first method is new customers can receive one NEWUSER10 discount, and the second method is claimed by loyalty cards. For loyalty card features, the customer will receive one loyalty stamp after return of each booking if the booking exceeds 12 hours. After collecting three stamps, they may claim the LOYALTY10 vouchers, which they can enjoy the discount on their future booking. For manual vouchers assignment, this only happens if there is any issue during booking and rewards for loyal customers. So, staff can also assign the vouchers to their account. On the staff side, they are also able to view the voucher's usage rate and history.

Furthermore, there is also car fleet management on the staff side. Staff is able to view the car details in the car list tab, such as the car plate, car model, price rate, features, last booked time and its status. Staff can directly manipulate the status using a dropdown, and also be able to edit the car details such as car picture and price. Staff is able to add new cars as well if there are any new leases in future. They will need to fill in the car plate, car model, car picture, price rate for 1 hours, 3 hours, 5 hours, 7 hours, 9 hours, 12 hours, and 24 hours. Staff are also able to view the past rental records of the cars for tracing purposes.

Additionally, there is customer management on the staff side. Staff can see all the details of each customer, such as their college, faculty, contact number, email, and status, and also can search for certain customers by input substring. The status of the customer also can be changed by dropdown, such as new, active, blacklisted, inactive. The customer is managed using soft delete instead of hard delete by setting the customer status to inactive so that our stakeholder can trace old customer information.

Moreover, there is rental management on the staff side. Staff can view all the rental details such as status, customer, car plate and booking period. Staff also can filter searching by customer name, status, car plate, month, or year. Meanwhile, admins can generate a report of rental by the filter as well. There is also a list of locations offered for pickup and return, and staff can view the delivery price for each location and be able to edit the price as well. Staff can also add a new location by filling in the location name and its delivery price.

Other than that, there is also payment management on the staff side. Staff can view all the transactions done by the customer, with the amount, rental duration and the receipt submitted. Certain transactions also can be filtered based on the customer's name, rental ID, year, or month. Meanwhile, admins can also generate a transaction report by the filter as well.

Now we move to the admin side. Admins have the privilege to view all staff and add new staff by inserting their information such as full name, email, position, and password. Admin also have the privilege to view analytics dashboard which show the graph of booking, revenue, customer gain, customer distribution by faculty and college and booking heatmap, filter by last 7 days, last 12 months and last 5 years.

1.1.3.2 Proposed System Flowchart

This section presents the proposed system workflows through flowcharts. An overall system flowchart gives a comprehensive view of the entire process flow, while six sub-flowcharts detail specific processes such as Activation, Booking, Pickup, Return, Staff Issue Return, and Late Return. These flowcharts are used to show the step-by-step procedures and decision-making processes within the system.

Figure below shows the main workflow of Hasta Car Rental System, starting from process user registration and login until the booking completion. It illustrates the main path for user roles including customer and staff and shows how different processes connect within the system.

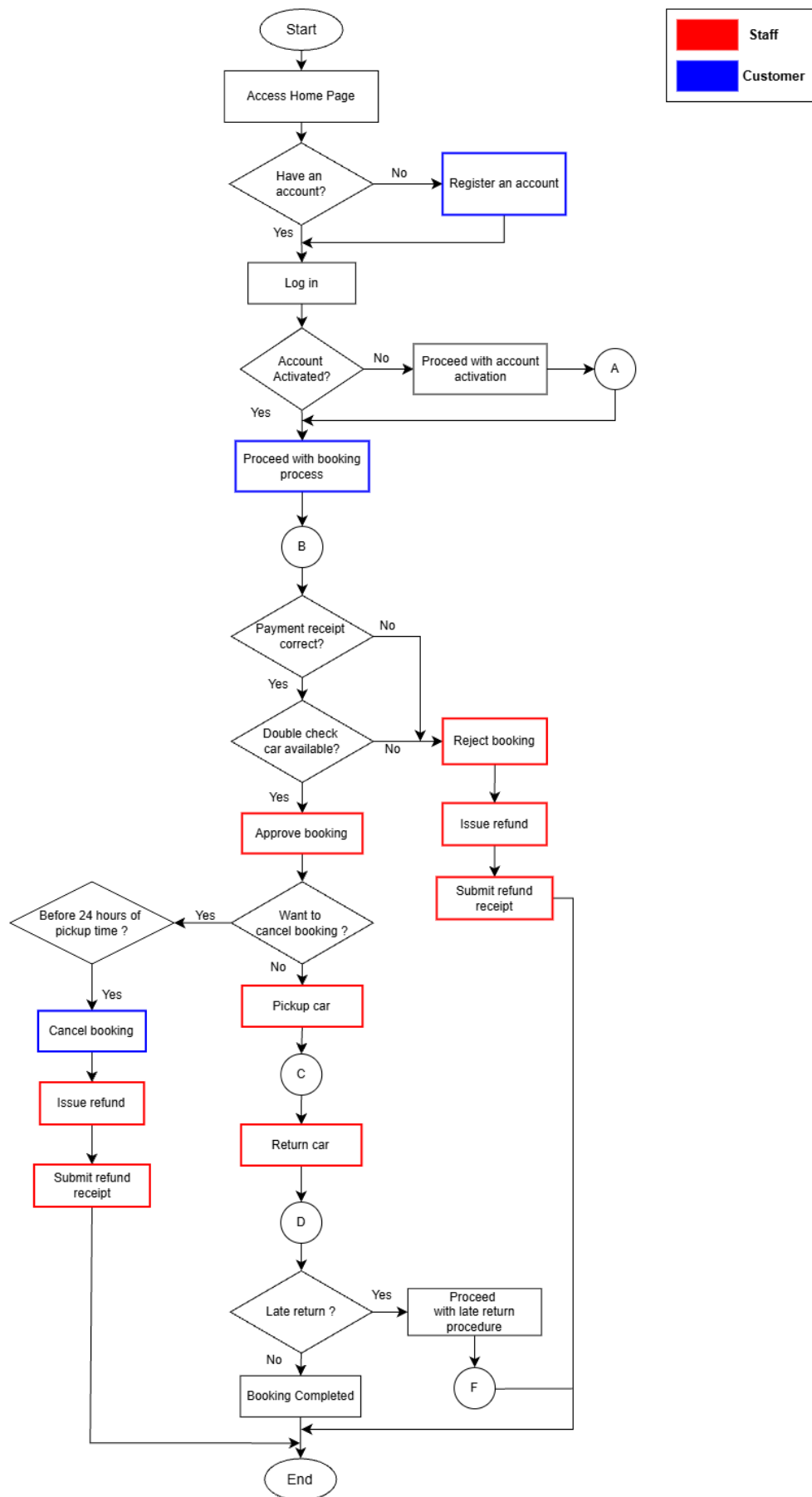


Figure below demonstrates the details of the account activation process. Customers need to provide all the personal information and submit required documents for staff to review them. Staff will approve or reject the activation after verifying the documents. Upon approval, customers can get full access to make booking. If the activation is rejected, customers can review the disapproval reason and submit a new activation request.

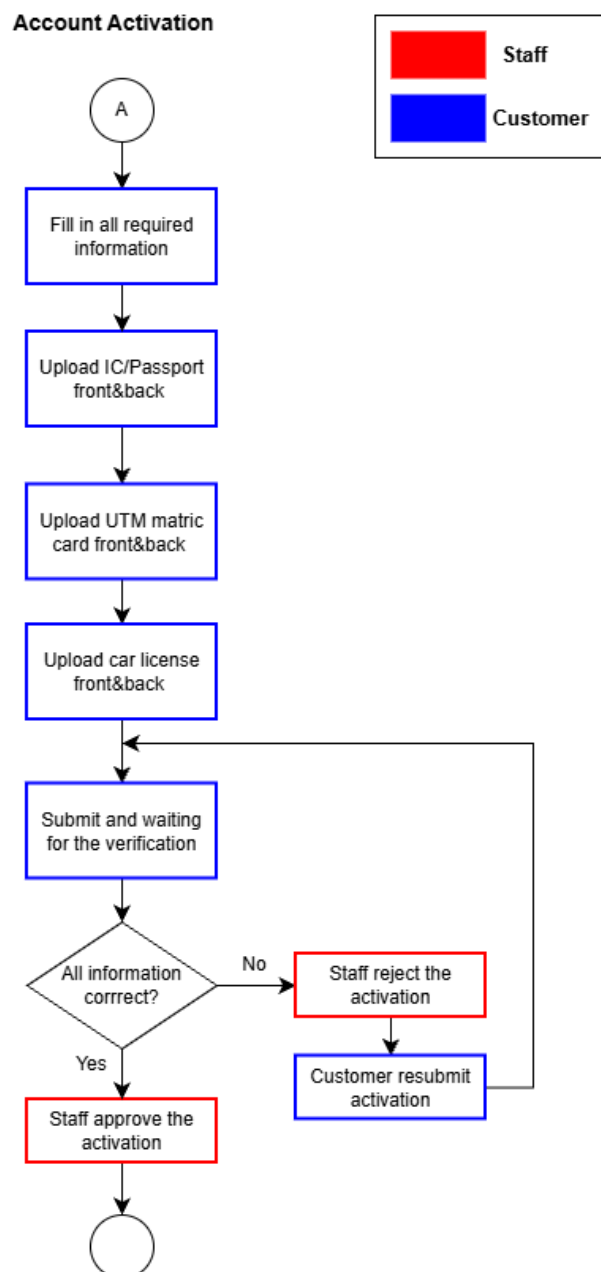


Figure below shows the booking process flows starting from select pickup and return details. After that, customers can select available cars that are filtered based on the booking details. A delivery fee will be required if the pickup or return or both are outside of UTM. Then customers can choose to use a voucher if available and complete the payment. The submitted booking needs to wait for staff to verify the payment and approve the booking.

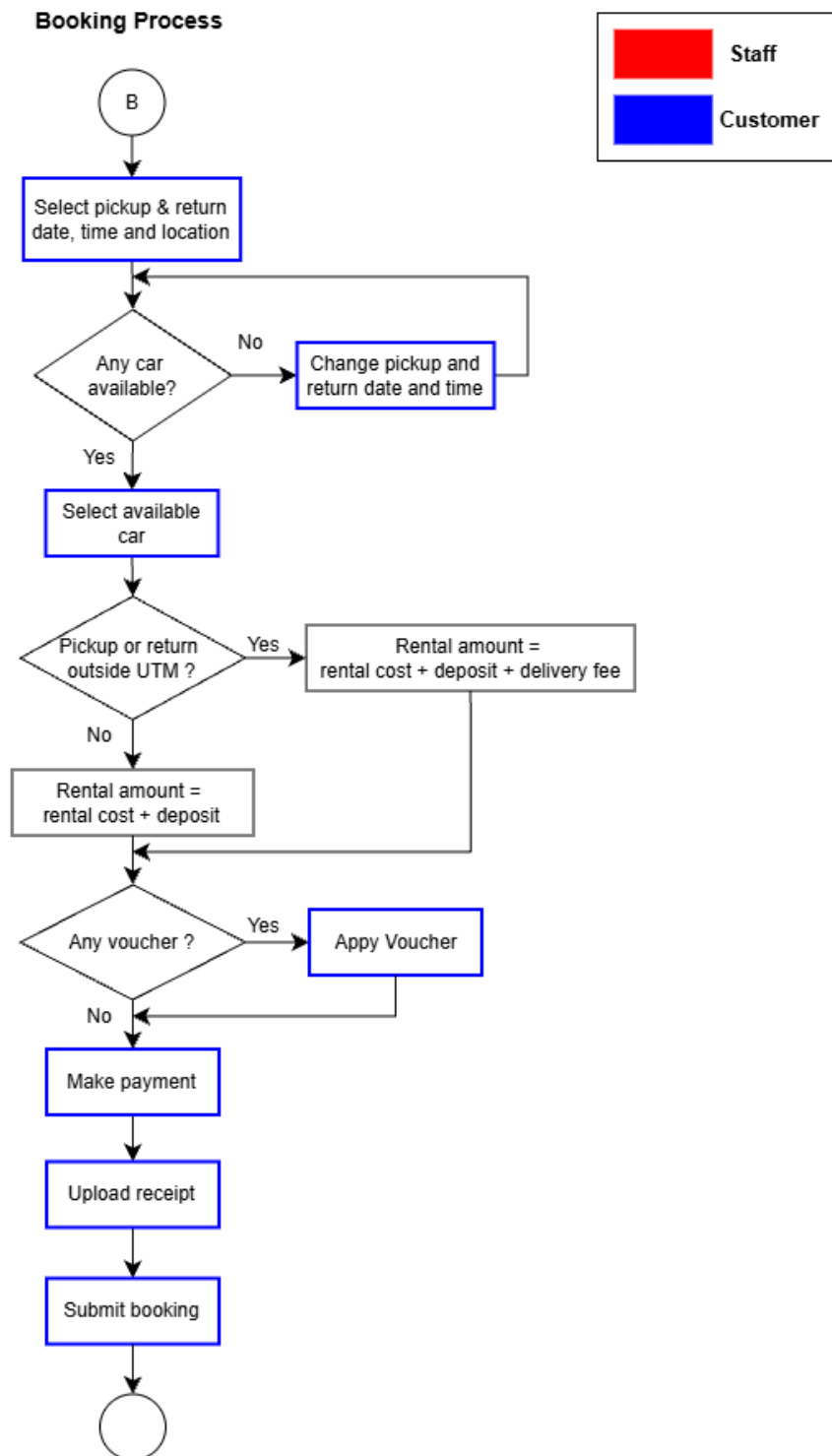


Figure below shows the car pickup process where customers can only pickup 30 minutes before the scheduled pickup time. Customers must download the agreement and inspection form, sign them, and then upload back to the system. Next, they need to upload five car photos including front, back, left, right view and the car odometer before submitting the form.

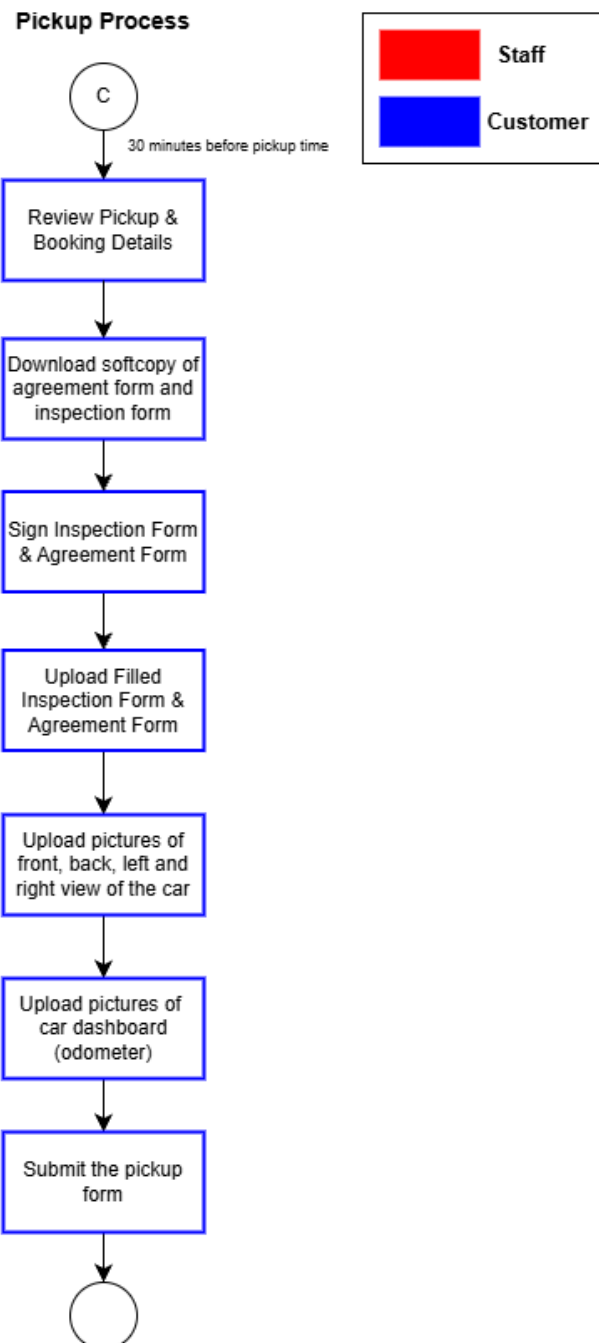


Figure below outlines the on-time car return process where customers need to upload six photos (including photo of car key), report an issue if faced any problem during rental, provide feedback and provide bank details in order to get the deposit refund. After the car return, there are some tasks needed to be completed by the staff including checking customer feedback, resolve issue if any, update car mileage and fuel bar, as well as issue deposit refunds.

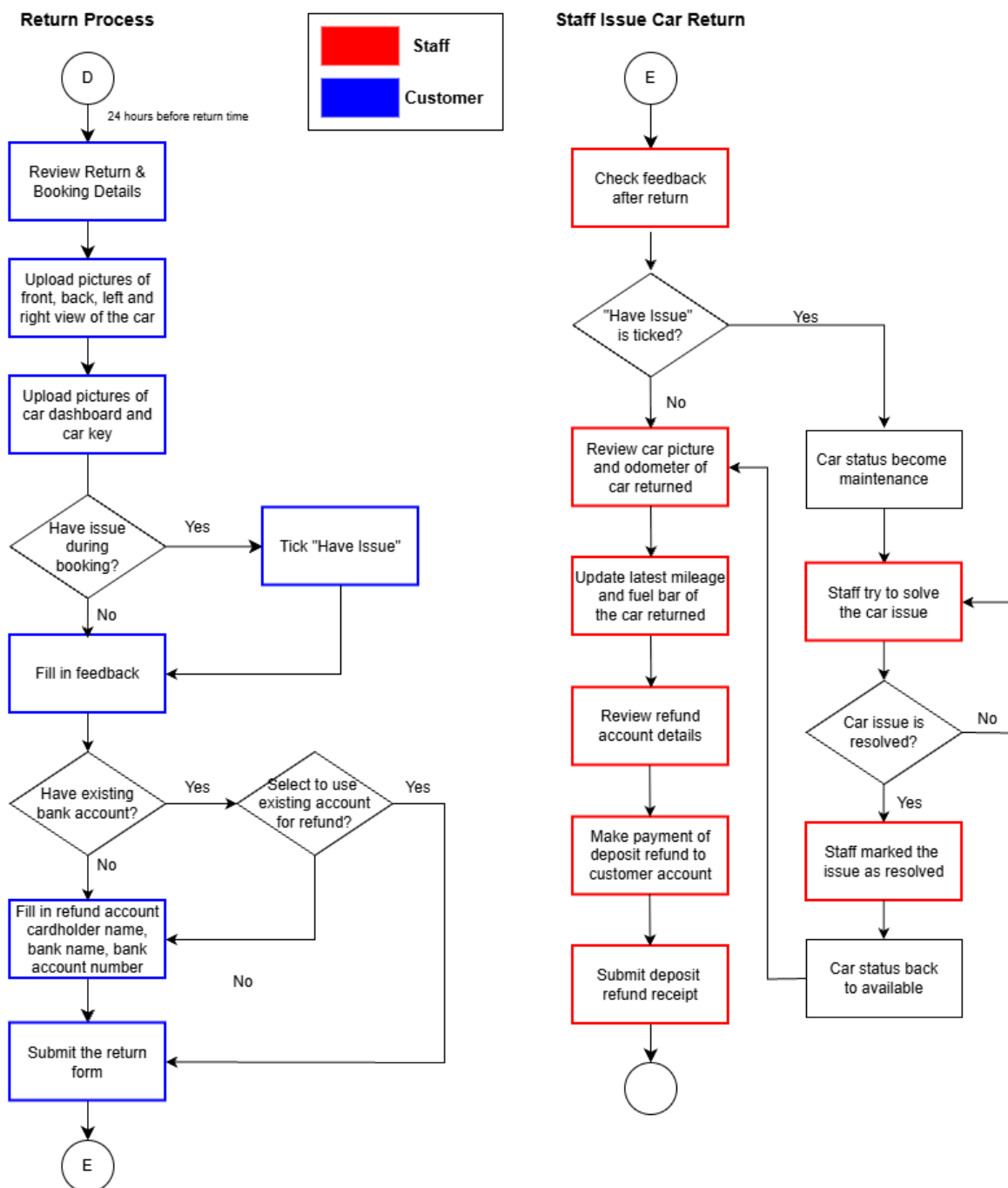
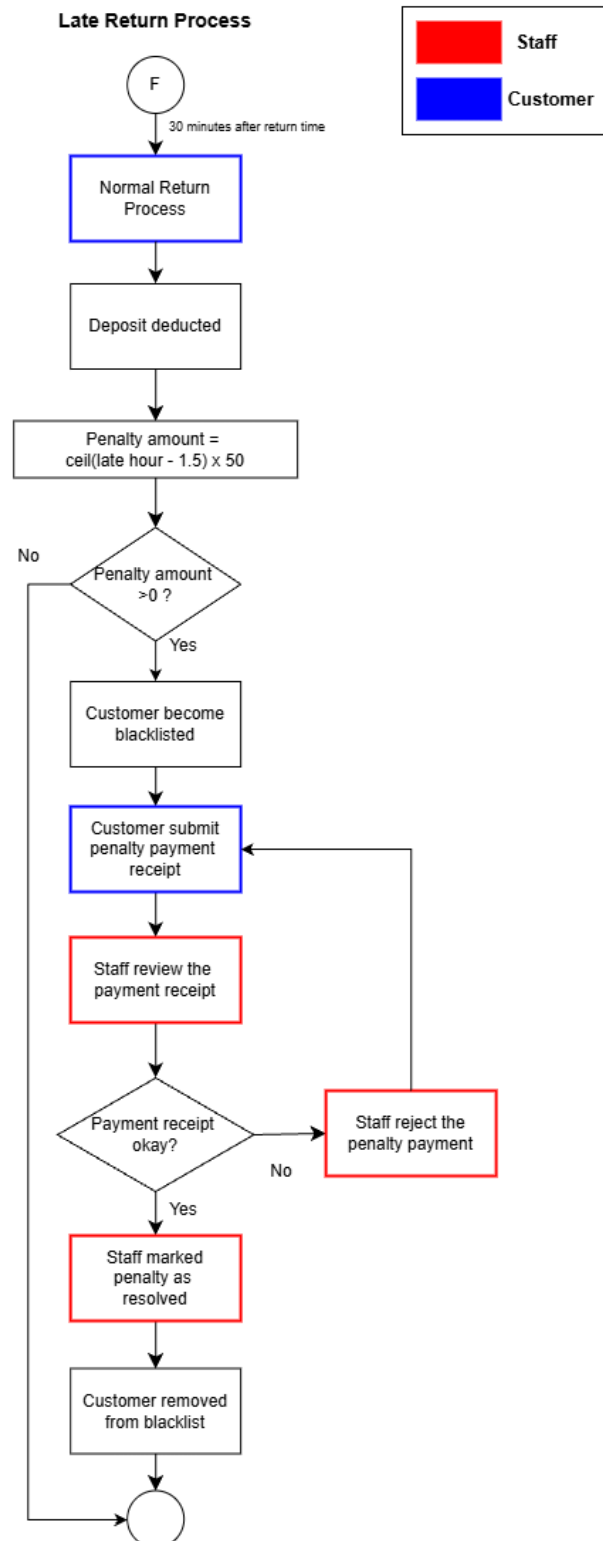


Figure below shows the late return process if the car is returned more than 30 minutes after the scheduled return time. Customers will be charged a penalty of RM50 per hour and their accounts will be automatically blacklisted. This causes them to not do any booking as their accounts are suspended until the penalty payment is approved by the staff.



1.1.3.3 Proposed System Sitemap

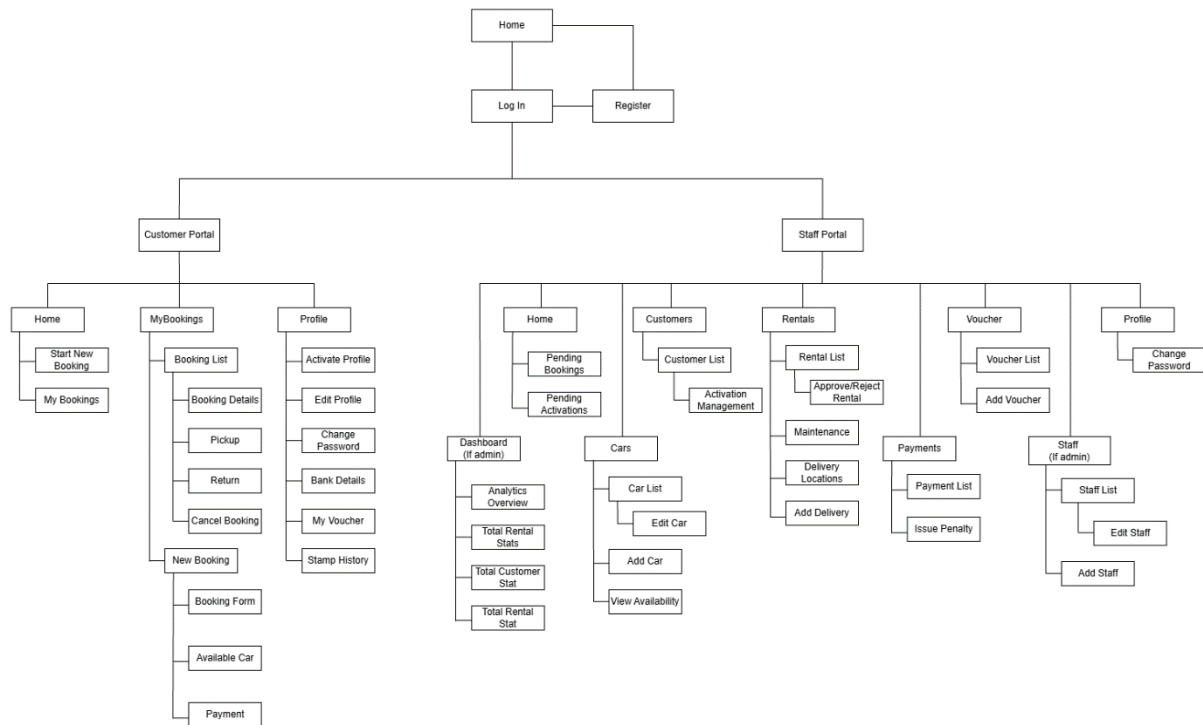


Figure above is the sitemap that shows the hierarchical structure and navigation flow of the Hasta Car Rental System. It shows all the accessible pages organized by user roles which are Customer, Staff and Admin.

1.2 DFD (TO-BE)

Figure 1 shows the Data Flow Diagram for TO-Be system with three entities and 10 processes. The customer entity represents the process flow that the customer can access. For the staff entity, it represents the process flow that the staff and admin can access, while for the admin entity, only the admin can access it.

Through the Manage Account process, customers can register, login, and activate their account with staffs' approval. The customer can then proceed with the Manage Booking process where they can make a booking request, cancel their booking, and submit their rental activity data for pickup and return. The staff can approve or reject the booking request. For the Manage Payment process, the customer needs to send their payment receipt and their bank details for deposit refund purposes. For blacklisted customers, they need to pay the penalty and the staff will verify the payment from their receipt. The staff and admin can update car information, customer information, location details, voucher details, and maintenance. However, only the admin can generate reporting and update staff information.

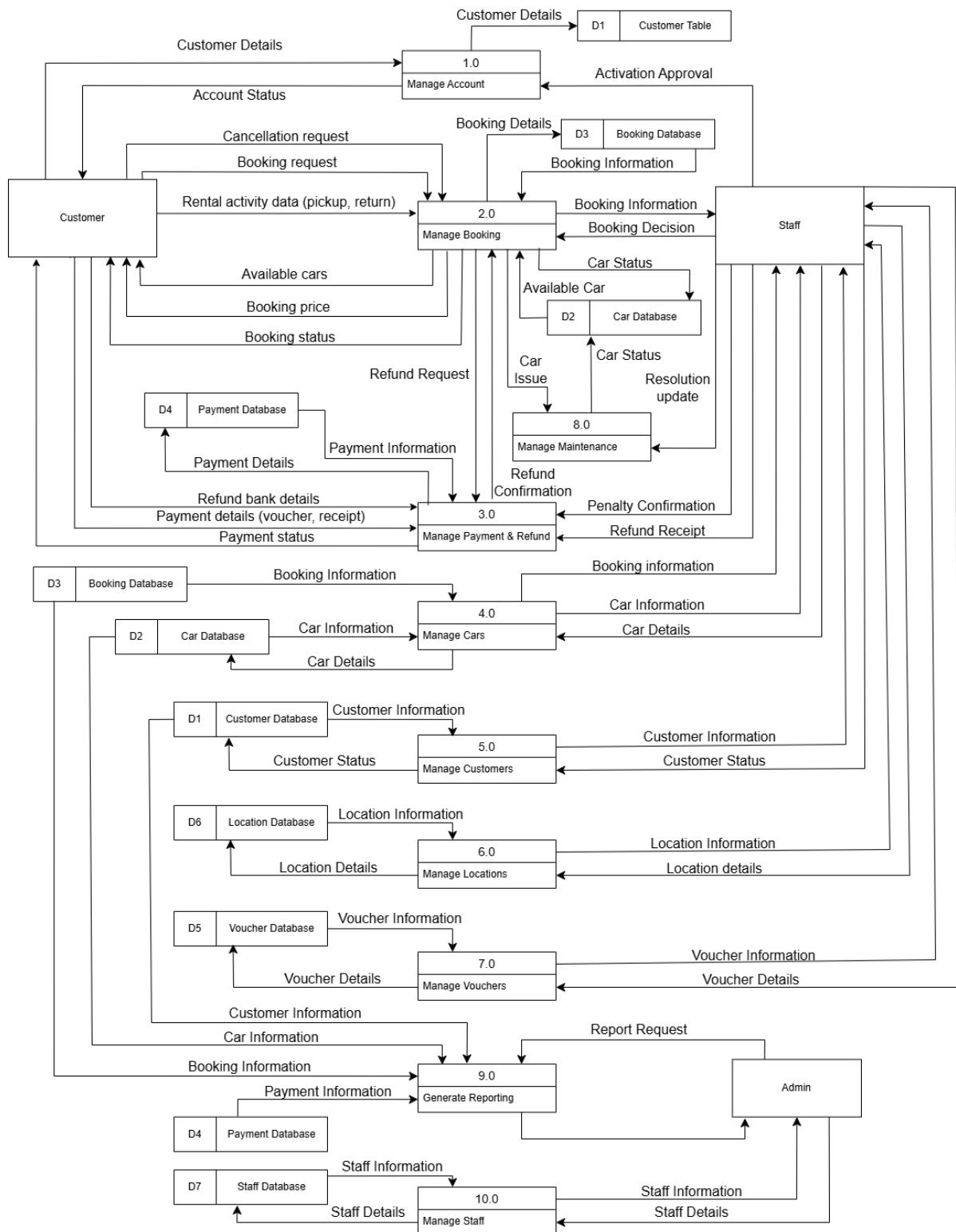


Figure 1: DFD of TO-BE System

1.3 Proposed Business Rule

(a) Customer and Activation Rules

1. Account activation rule:

Before a customer can make a booking, the account needs to be activated such that the status = 'Active'.

2. Blacklist restriction rule:

A customer with status = 'Blacklisted' will be blocked to access the system.

3. Activation rejection reason rule:

When an activation is rejected, a rejected reason must be provided by staff.

4. Duplicate activation rule:

A customer cannot submit a new activation request while having a pending activation.

(b) Car Availability Rules

1. Car availability rule:

A car can only be booked if there is no overlapping rental for the selected pickup–return date/time.

2. Return period cooldown rule:

A car becomes available only after 1 hour of the rental's return time.

3. Maintenance exclusion rule:

A car with status 'Maintenance' will not be displayed in the available car list.

(c) Booking Rules

1. Booking rejection rule:

When staff reject a booking, the car's status is reset to 'Available' with a disapprove reason.

2. Booking period validation rule:

The date and time of return must be after the date and time of pickup.

(d) Cancellation Rules**1. 24-hour cancellation rule:**

A booking can be cancelled only if there is more than 24 hours left before the scheduled pickup time and the status of that booking is either “Pending” or “Approved”.

(e) Payment and Refund Rules.**1. Refund bank details rule:**

In order to process a refund, bank details must be provided.

2. Rejection refund rule:

When a booking is rejected or cancelled, a full refund will be issued by staff.

(f) Voucher Rules**1. New user voucher rule:**

A 'NEWUSER10' voucher is automatically given to a customer upon registration.

2. Voucher validity rule:

A voucher can only be applied if usage = 'Unused' and expiry_date >= current date.

3. Voucher usage update rule:

Once a voucher is applied to a rental, its usage is updated to 'Used'.

4. Loyalty voucher claim rule:

A customer can claim a 'LOYALTY20' voucher only when they have at least 3 available loyalty stamps.

(g) Late Return and Penalty Rules**1. Late return cooldown period rule:**

A return is considered late only if it exceeds 30 minutes past the scheduled return time.

2. Late return penalty rule:

For late returns after the cooldown period, a penalty of RM50 per hour (minus RM50 deposit) will be charged.

3. Blacklist on penalty rule:

If a customer has a late penalty amount greater than zero, their status is set to 'Blacklisted'.

4. Deposit forfeiture rule:

The RM50 deposit is forfeited if the customer returns late.

(h) Loyalty Stamp Rule**1. Loyalty stamp rule:**

A customer can earn 1 loyalty stamp for each completed rental that is more than 12 hours.

(i) Car Maintenance Rule**1. Maintenance rule based on issues:**

When a customer reports an issue during car return, the car's status is changed to 'Maintenance'.

1.4 Database System Requirements

1.4.1 System Overview

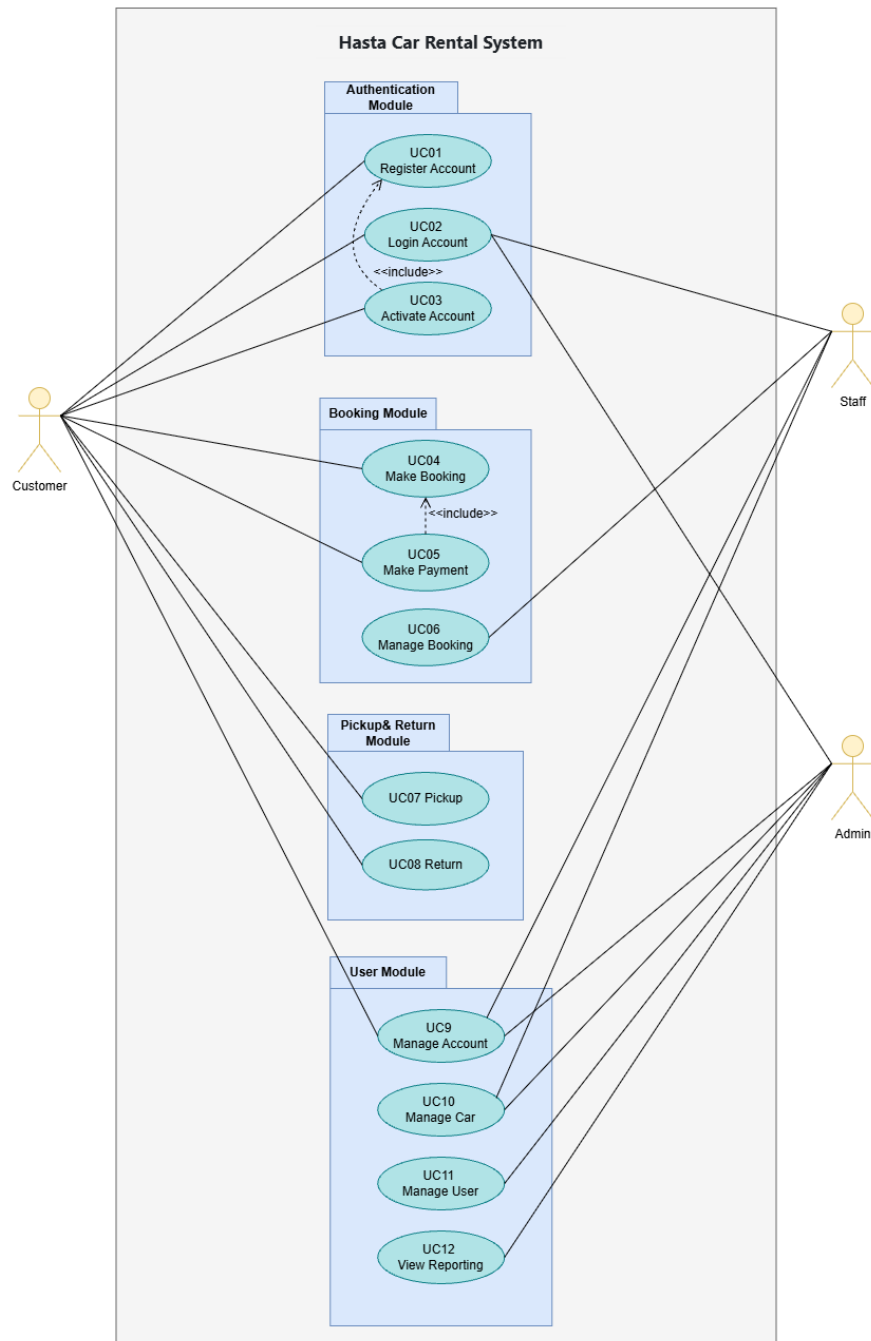


Figure 2: Use Case Diagram

Figure 2 above illustrates the use case diagram for the Hasta Car Rental System with three actors: Customer, Staff, and Admin. Customers can register, login, activate accounts, make bookings, submit payments, pickup and return vehicles, and manage their accounts. Staff handle account activation, booking management and car management. Admins manage users, cars, and view system reports.

Use Case	Function	Description
UC01	Register Account	This use case allows customers to sign up as a user for the system.
UC02	Login Account	This use case allows users to log in to the system using their email and password.
UC03	Activate Account	This use case allows customers to activate accounts by submitting required documents then staff reviews and approve activations.
UC04	Make Booking	This use case allows customers to create a booking by specifying pickup and return details and selecting available cars.
UC05	Make Payment	This use case allows customers to complete payment by uploading the payment receipt and submitting booking.
UC06	Manage Booking	This use case allows staff and admin to view all bookings, approve or reject the bookings.
UC07	Pickup	This use case allows customers to process car pickup by completing the agreement and inspection form and capturing pickup car photos.
UC08	Return	This use case allows customers to return the car by capturing return car photos including car key photo, providing feedback and reporting an issue that occurred during the rental period (if any).
UC09	Manage Account	This use case allows all users to view their own profile information and change account password.
UC10	Manage Car	This use case allows staff and admin to add new vehicles to the rental fleet and update existing car information.
UC11	Manage User	This use case allows the admin to view all user accounts in the system, create new accounts for staff and update user status.
UC12	View Reporting	This use case allows admin to view comprehensive reports including rental statistics, revenue reports and customer analysis.

1.4.2 Module <Authentication>

1.4.2.1 Module Description

Use Case	Function	Description
UC01	Register Account	This use case allows customers to sign up as a user for the system.
UC02	Login Account	This use case allows users to log in to the system using their email and password.
UC03	Activate Account	This use case allows customers to activate accounts by submitting required documents then staff reviews and approve activations.

This module handles the account creation and account authentication, which is the most basic component in a digitalized booking system. At first, user need to register an account first, then login using the credentials. For a new account, booking feature is still disabled because customer have not activated their account yet. Hence, they have to activate their account by filling in required information and submitting required documents, and staff need to review the activation requests and choose to approve or reject the activations.

1.4.2.2 Transaction Requirement: Data Entry

- Register new account by customer
- Create staff account by admin
- Submit activation request with uploaded documents
- Auto-issue NEWUSER10 voucher upon registration

1.4.2.3 Transaction Requirement: Data Update/Deletion

- Update user password
- Update customer status to “Approved” or “Reject” after review activation request
- Update customer status via dropdown (New, Pending, Active, Inactive, Blacklisted)
- Update staff information (Full name, email, position)
- Delete staff account (Soft Delete)

1.4.2.4 Transaction Requirement: Data Queries

- View pending activation requests
- Retrieve customer details and uploaded documents
- Retrieve user role for dashboard redirection

1.4.2.5 Local Conceptual Data Model (ERD)

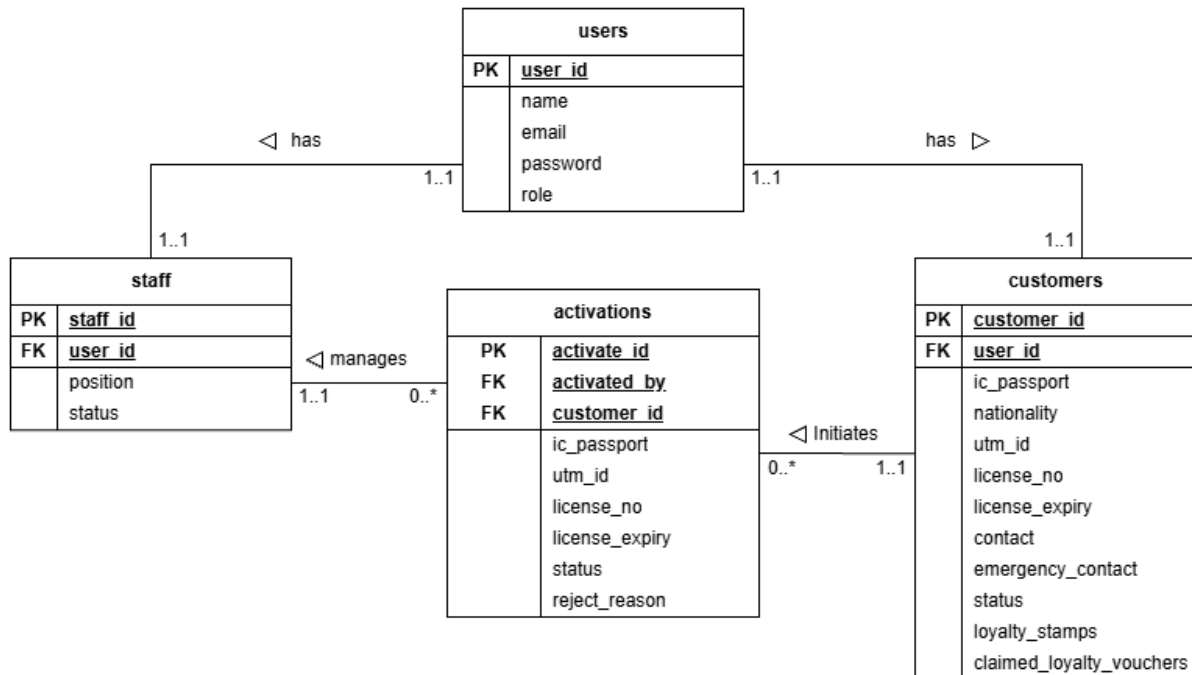


Figure above shows the entities and relationships that exist in the Authentication Module of the Hasta Car Rental System. The module involves four key entities which are users, customers, staff, and activations. The users entity serves as the parent table that contains the basic login credentials and role of each user. Customers and staff are the entities that extend from the users entity which store specific information of each role. The activations entity represents the process of tracking customer account activation requests and establishing a relationship between customers and staff. This module handles user registration, login and then account activation process.

1.4.3 Module <Booking>

1.4.3.1 Module Description

Use Case	Function	Description
UC04	Make Booking	This use case allows customers to create a booking by specifying pickup and return details and selecting available cars.
UC05	Make Payment	This use case allows customers to complete payment by uploading the payment receipt and submitting booking.
UC06	Manage Booking	This use case allows staff and admin to view all bookings, approve or reject the bookings.

This module is the most crucial component of the digitalized booking system. At first, customer have to make booking by filling in the pickup and return details, select car, then they need to make payment by uploading the payment receipt and submit booking request. For now, the booking is still pending, and staff need to manage the booking. Staff can review the booking details and choose to approve or reject the booking.

1.4.3.2 Transaction Requirement: Data Entry

- Create new booking with selected dates, time, locations and car
- Submit payment receipt
- Apply voucher to booking
- Set initial booking status to “Pending”

1.4.3.3 Transaction Requirement: Data Update/Deletion

- Update booking status to “Approved” or “Rejected” after review booking detail and payment receipt
- Update car status to “Rented” if approved
- Upload refund receipt for rejected and cancelled booking
- Cancelled booking and update booking status to “Cancelled”
- Update voucher usage to “Used”

1.4.3.4 Transaction Requirement: Data Queries

- Check car availability for selected dates and time
- Retrieve car details
- View all pending bookings
- Retrieve booking details for review
- View all payment list

1.4.3.5 Local Conceptual Data Model (ERD)

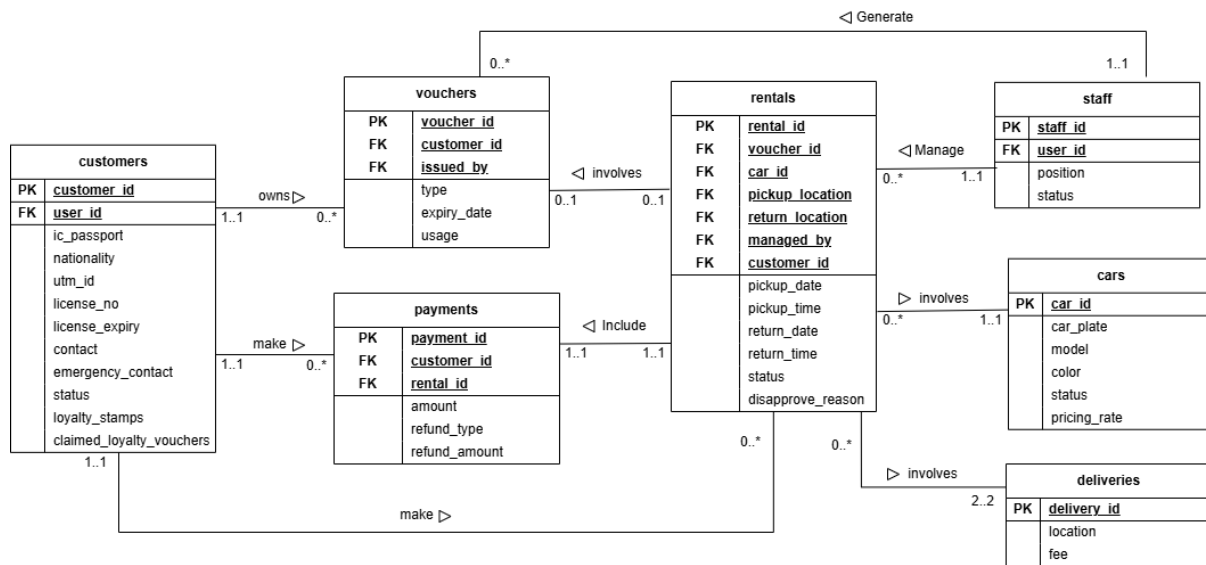


Figure above represents the entities and relationships in the Booking Module, which handles the entire booking process. The main entity is rentals which connect to multiple entities: customers (persons making the bookings), cars (vehicles being booked), staff (persons handling the bookings), vouchers (optional booking discounts), payments (payment processes for bookings), and deliveries (pickup and return locations). This module implements a composition relationship between payments and rentals which show that payments cannot exist independently without rentals. Moreover, vouchers are owned by customers, and it has a one-to-one relationship with the rentals entity.

1.4.4 Module <Pickup & Return>

1.4.4.1 Module Description

Use Case	Function	Description
UC07	Pickup	This use case allows customers to process car pickup by completing the agreement and inspection form and capturing pickup car photos.
UC08	Return	This use case allows customers to return the car by capturing return car photos including car key photo, providing feedback and reporting an issue that occurred during the rental period (if any).

This module is also very crucial to complete the booking process. When it is 30 minutes before pickup time, the pickup is available, and customer need to process pickup by submitting inspection form and agreement form and uploading car photos. Afterwards, rental will become Picked Up. For return, customer can return before 24 hours before return time. They need to upload car photos again, provide feedback, report any issue during feedback and fill in refund details. After car is returned, staff need to update car mileage and fuel bar, issue feedback, do manual inspection, and issue refund by submitting refund receipt.

1.4.4.2 Transaction Requirement: Data Entry

a) Pickup Process

- Upload signed agreement and inspection form
- Upload 5 car photos including front, back, left, right and odometer
- Upload additional photos if found damage on the car (optional)
- Record pickup timestamp

b) Return Process

- Upload 6 car photos (front, back, left, right, odometer, car key)
- Report issues via checkbox
- Enter customer feedback
- Enter bank details for RM50 deposit refund
- Record return timestamp

1.4.4.3 Transaction Requirement: Data Update/Deletion

a) Pickup Process

- Update booking status to “Picked Up”

b) Return Process

- Update booking status to “Returned” or “Late Returned”
- Update car status to “Maintenance” if issues reported, else update to “Available”
- Update customer status to “Blacklisted” if the car is late returned
- Update car mileage and fuel level
- Mark car issues as resolved
- Update car status back to “Available” after maintenance
- Upload deposit refund receipt
- Update booking status to “Completed”
- Add loyalty stamp if booking duration > 12 hours

1.4.4.4 Transaction Requirement: Data Queries

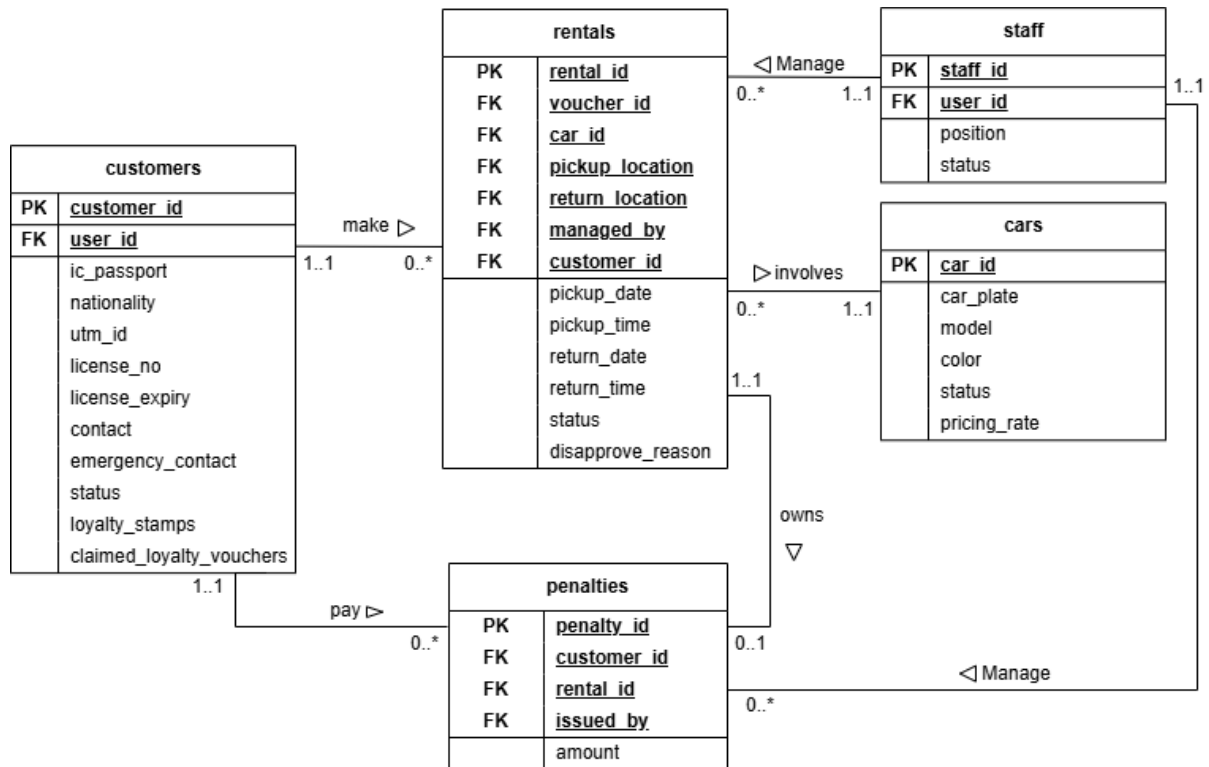
a) Pickup Process

- Retrieve booking details
- Check if before 30 minutes of scheduled pickup time
- View upload forms and car photos

b) Return Process

- Retrieve booking details
- Check expected return time
- Check if return is late (>30 minutes of the scheduled return time)
- Calculate late penalty (RM50 per hour)
- View customer feedback and reported issues
- View uploaded return photos
- View cars flagged for maintenance

1.4.4.5 Local Conceptual Data Model (ERD)



This diagram represents the Pickup and Return Module which relates to the handling of collecting and returning vehicles process. This module emphasizes the rentals entity and the link it has with the customers who pick up and return the cars, cars that the customers collect and return, staff who handle the return tasks and penalties and also the penalties that the customers need to pay for late return. A composition relationship exists here between the rentals and the penalties, showing that the records of the penalties are closely linked to their parent, the rental transaction. This module deals with the collection of returns, return details such as feedback and issues, tracking of loyalty stamps, late returns, penalties, and vehicle maintenance issues reported.

1.4.5 Module <User>

1.4.5.1 Module Description

Use Case	Function	Description
UC09	Manage Account	This use case allows all users to view their own profile information and change account password.
UC10	Manage Car	This use case allows staff and admin to add new vehicles to the rental fleet and update existing car information.
UC11	Manage User	This use case allows the admin to view all user accounts in the system, create new accounts for staff and update user status.
UC12	View Reporting	This use case allows admin to view comprehensive reports including rental statistics, revenue reports and customer analysis.

This module acts as a glue between all the modules. Customer can manage their own account, and staff can manage their account as well. Staff can also manage car, such as update its status or add new car. Staff can also manage user by viewing user list and update their status. Admin can also view reporting such as rental statistics, revenue reports and customer analysis, and export them into PDF format or CSV format.

1.4.5.2 Transaction Requirement: Data Entry

a) Penalty Process

- Submit penalty payment receipt by customer

b) Car Management Process

- Add new car to fleet with details
- Upload car photo
- Set pricing rates
- Set special features if any

c) Reporting & Monitoring Process

- No data update or deletion are included in this process.

1.4.5.3 Transaction Requirement: Data Update/Deletion

a) Penalty Process

- Approve penalty payment by staff
- Update customer status from “Blacklisted” to “Active”

b) Car Management Process

- Edit car details
- Update car pricing rates
- Change car status via dropdown (Available, Rented, Maintenance, Inactive)

c) Reporting & Monitoring Process

- No data update or deletion are included in this process.

1.4.5.4 Transaction Requirement: Data Queries

a) Penalty Process

- View pending penalty payments
- View penalty payment receipt
- Check customer status

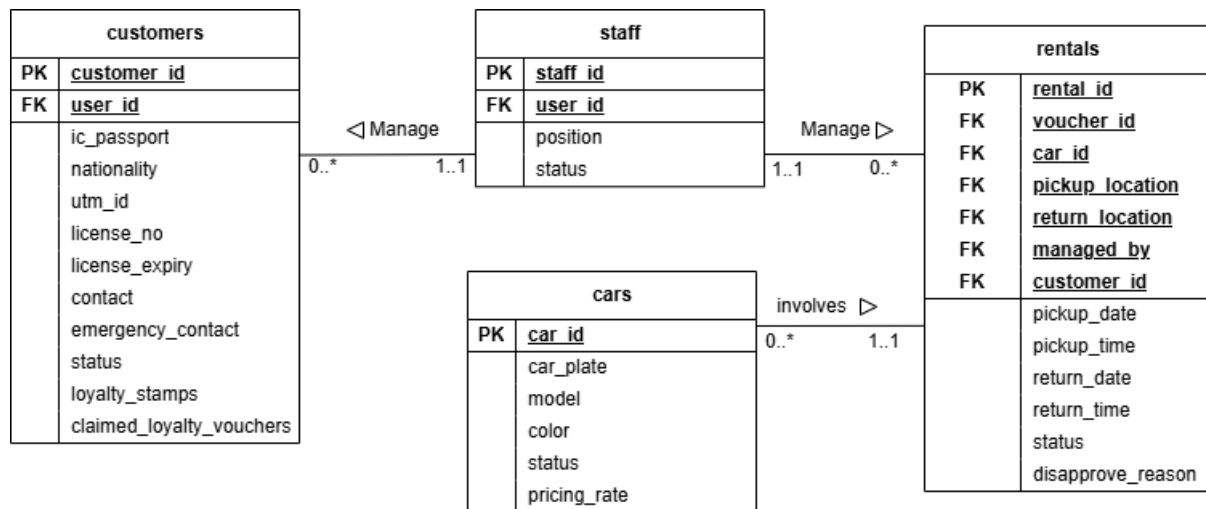
b) Car Management Process

- View complete car list
- Check car availability for specific dates

c) Reporting & Monitoring Process

- Rental report
- Revenue report
- Booking trends graph
- Customer distribution charts by faculty and college
- Booking heatmap
- Filter analytics by time period
- Export reports to PDF/Excel
- View dashboard key metrics (total bookings, revenue, customers)

1.4.5.5 Local Conceptual Data Model (ERD)



In this diagram, there is representation of entities and relationships of the User Module, which is responsible for account management, car fleet management, as well as managing a system on a whole. In the User Module, there is a users entity with a connection to customers & staff entities, so users can handle their respective profiles. Staff members are responsible for handling rentals in their organizations, which is indicated by the relationship between staff & rentals entities. In car fleet management, there is a cars entity responsible for storing data related to cars & their availability. This module enables users to manage their own profiles, allows staff to manage the car fleet, enables admins to manage user accounts and staff members, and supports reporting and analytics through access to rental and customer data.

1.5 Global Conceptual Data Model (Global ERD)

1.5.1 Global Conceptual Design (ERD)

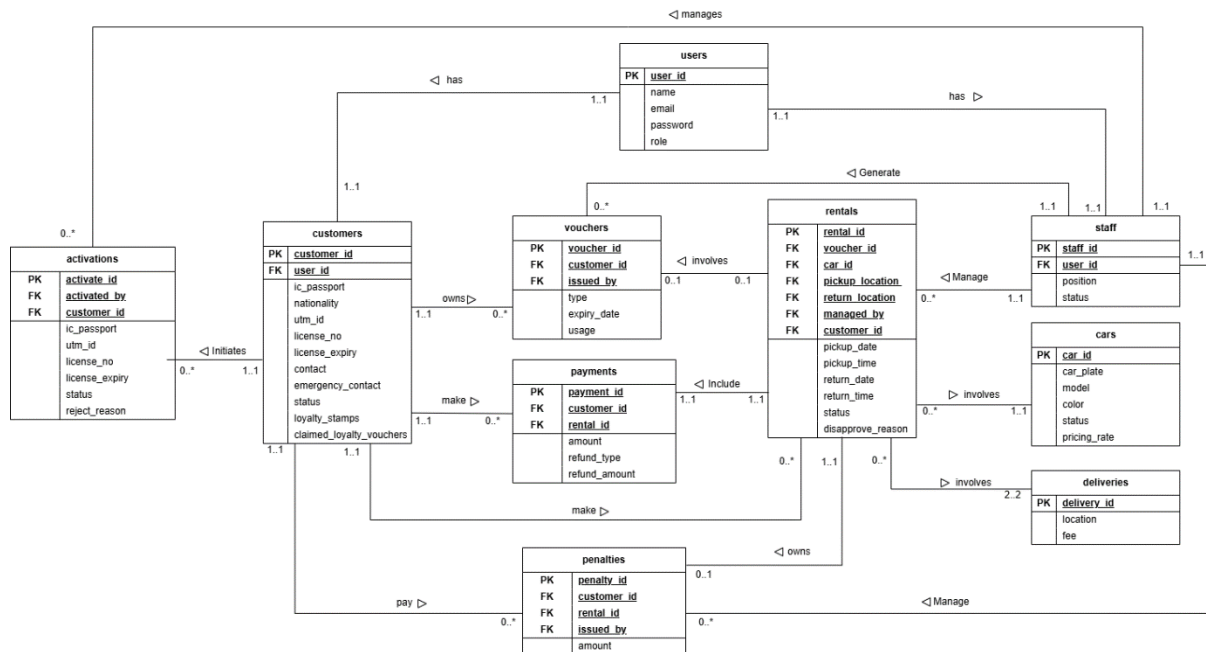


Figure above integrates all entities and relationships from the four modules which are Authentication, Booking, Pickup and Return, and User. The Users entity sits at the top of the hierarchy, responsible for authentication for all types of users. Customers will initiate multiple processes: activations, bookings or rentals, payments, and voucher ownership, while also being subject to penalties. Rentals serve as the central object that connects customers, cars, staff management, vouchers, payments, deliveries, and penalties. Composition relationships are shared between Rentals-Payments and Rentals-Penalties, showing strong lifecycle dependencies. Aggregation relationships show that Customers own multiple related records for each type of information, such as activations, rentals, payments, vouchers, penalties, which can exist independently. The diagram now presents a complete data flow throughout and interdependencies among all system modules to give an idea of how varied entities interact over a car rental business process from registration to the completion of a rental.

1.5.2 Enhanced ERD (EERD)

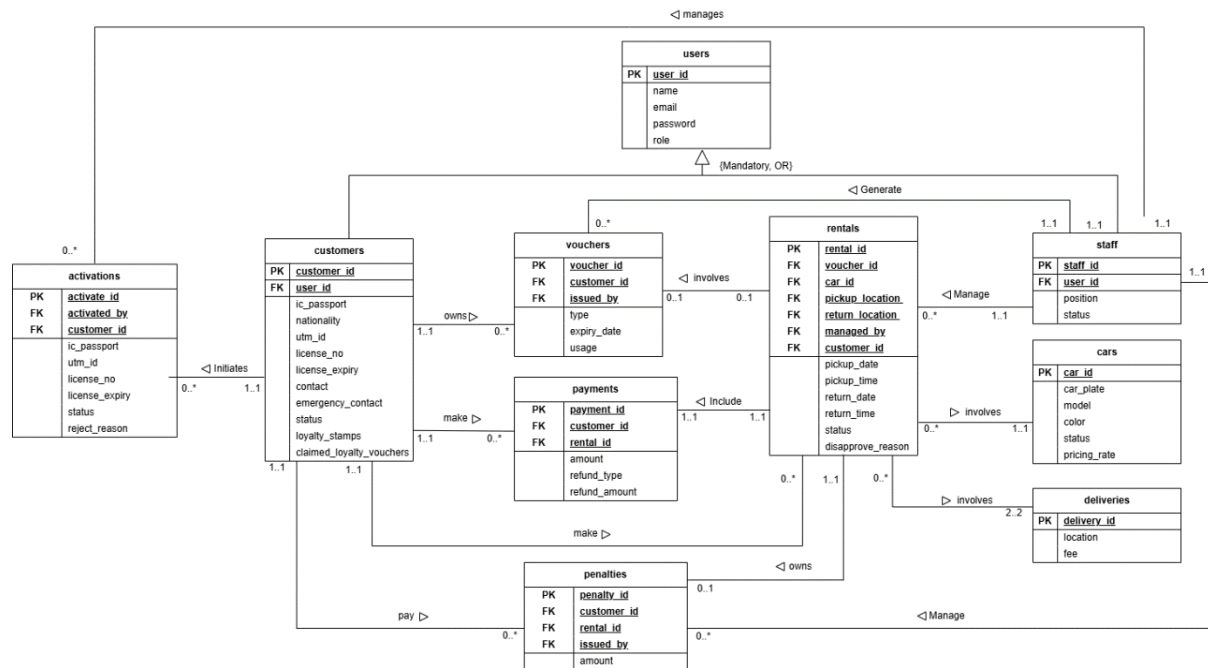
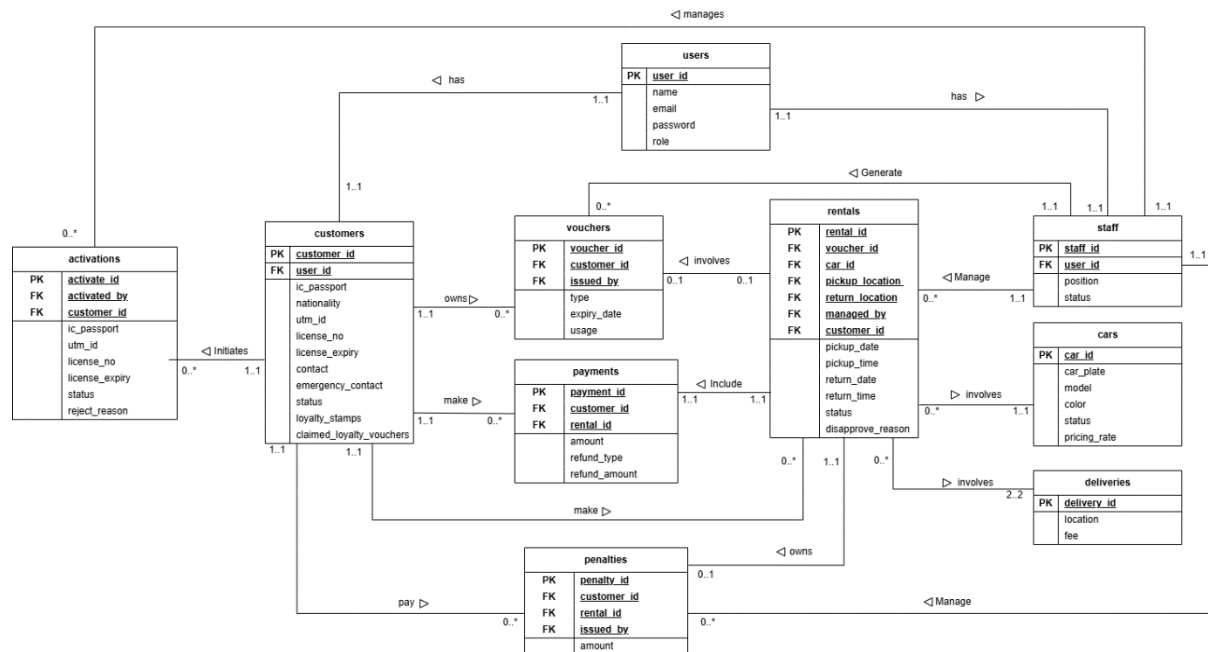


Figure above shows the conceptual EERD that represents how generalization and specialization exist in the Hasta Car Rental System. The Users entity is presented as a super class with Customer and Staff entities acting as its sub classes. The practice is one way of showing inheritance, where both entities have attributes common to them, referred to as (user_id, name, email, password, role), while having other attributes specific to their entities.

1.5.3 Verified Global Conceptual Design (Verified ERD)



The global conceptual ERD is then reviewed by data administrators to verify and validate with the business rules. After reviewing, it is found out that the global conceptual ERD produced had met the business rules and does not disobey any data integrity. Therefore, the verified global conceptual ERD does not change any aspects of global conceptual ERD produced. Therefore, it is already ready to be converted into relational tables.

1.5.4 Data Dictionary of Global Conceptual ERD

1. Users

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each user, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
name	varchar(50)	NOT NULL	Full name of user (NRIC), uses size of 50 because it is sufficient to store all names.
email	varchar(255)	NOT NULL, UNIQUE	Email address of the users to be used to login, uses varchar 255 as it is the industry standard for email fields. Each email only can be used to create one account so it must be unique.
role	varchar(20)	NOT NULL, DEFAULT 'customer'	Role of user in the system to manage user views and system boundaries. (e.g. staff, admin, customer) Uses varchar 20 as it is sufficient for any role name. Any account registration will create a user with the role of customer, only admin can create user with role of staff.
password	varchar(255)	NOT NULL	This is to store the user passwords for account login credentials. Passwords will be hashed using bcrypt to avoid brute forcing to crack passwords. VARCHAR(255) is safe and recommended for any potential changes to the hashing algorithm or if the library adds extra metadata to the string, such as Spring Security, which uses a {bcrypt} prefix.

2. Customers

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each customer, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
user_id (FK)	bigint	FOREIGN KEY, UNIQUE, NOT NULL	Links to the user account to inherit the attributes from user.
ic_passport	varchar(20)	NULLABLE	IC (for Malaysian) or passport number (for foreigner) of the customer. It is null when the account is registered, and filled after the account is activated. VARCHAR 20 is used because it is sufficient to occupy all IC and passport numbers in world.
nationality	varchar(50)	NULLABLE	Nationality of customer, this is to form a composite key with ic_passport, as a customer from one country might have the same passport number with another customer from another country, but each country only can have one distinct passport number. It is null when the account is registered, and filled after the account is activated. Varchar 50 can be used for most of the country.
utm_id	varchar(20)	UNIQUE, NULLABLE	UTM student card or staff card number, it must be unique as one student/staff only can have one account, and varchar 20 is used as it's sufficient for all UTM ID (e.g. A24CS0001) It is null when the account is registered, and filled after the account is activated.

contact	varchar(20)	NULLABLE	Phone contact number of customer, using varchar 20 because it is enough for all telephone numbers. It is null after registration, and filled after the account is activated.
emergency_contact	varchar(20)	NULLABLE	Emergency phone contact number of customer, using varchar 20 because it is enough for all telephone numbers. It is null when the account is registered, and filled after the account is activated.
license_no	varchar(20)	NULLABLE	The car license number of customers as their eligibility to book a car, use varchar 20 as it is enough for all the licences. It is null when the account is registered, and filled after the account is activated. License number is not unique, as different countries might produce the same license no.
license_expiry	date	NULLABLE	The expiry date of car license. It is null when the account is registered, and filled after the account is activated.
college	varchar(30)	NULLABLE	The residential college of students or staff in UTM. For students/staff that live outside, their college will be “NON APPLICABLE”, else it will be UTM college such as KTDI. Varchar 30 is used as it is enough for all college names. It is null when the account is registered, and filled after the account is activated.

faculty	varchar(30)	NULLABLE	The faculty of the customer such as Faculty of Computing, Faculty of Civil Engineering, etc. It is null when the account is registered, and filled after the account is activated. Varchar 30 is used as it is enough for all faculty names.
address	varchar(255)	NULLABLE	The home address of the customer, varchar 255 is enough to occupy most of the address. It is null when the account is registered, and filled after the account is activated.
bank_name	varchar(20)	NULLABLE	The bank name of the customer bank account, such as RHB, Maybank, etc. This is used for refund purposes, and customers can decide to fill it or not in profile. Varchar 20 is used as it is enough for all bank names.
bank_account	varchar(20)	NULLABLE	The bank account number of the customer bank. This is used for refund purposes, and customers can decide to fill it or not in their profile page, so it is nullable in constraint. Varchar 20 is used as it is enough for account numbers.
status	varchar(20)	NOT NULL, DEFAULT 'New'	The status of the customer, by default the customer is New. Varchar20 is enough for all status (Active, Inactive, Blacklisted)
loyalty_stamps	int	NOT NULL, DEFAULT '0'	The current loyalty stamps owned by customers.
claimed_loyalty_vouchers	int	NOT NULL, DEFAULT '0'	The total loyalty vouchers claimed by customers to track how much stamps are used and how much vouchers obtained.

3. Staff

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each staff, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
user_id (FK)	bigint	FOREIGN KEY, UNIQUE, NOT NULL	Links to the user account to inherit the attributes from user.
position	varchar(20)	NOT NULL, DEFAULT 'Staff'	The position of staff for managing the privilege and accessibility of certain functions. For example, staff can only do tasks, while admin can view analytics dashboard and download reporting.
status	varchar(20)	NOT NULL, DEFAULT 'Active'	The status of staff, e.g. Active or Inactive, this is important for the soft delete feature to trace back previous staff.

4. Cars

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each vehicle, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
car_plate	varchar(20)	NOT NULL, UNIQUE	Plate number for each vehicle, it must be unique as there are no two vehicles with same plate number, varchar 20 is safe to cover all car plate variation.
model	varchar(30)	NOT NULL	The model name of the car, such as Proton X70, etc. Varchar 30 is just enough for a simple model name.
color	varchar(20)	NOT NULL	The color of the car, such as Gold, White, Black, etc. Varchar 20 is totally enough for all color names.
status	varchar(20)	NOT NULL, DEFAULT 'Active'	The status of the car, by default the car is Active, cars that are soft deleted will become Inactive, cars under booking will become Rented, and cars having issues will become Maintenance.
mileage	int	NOT NULL	The mileage of the car, updated by staff after the customer returns the car.
fuel_bar	int	NOT NULL	The fuel bar of the car, updated by staff after the customer returns the car.

pricing_rate	json	NOT NULL	The list of pricing rates of car rental for each hour. JSON is used as Hasta has a special way to calculate price, which they provide pricing rate of 1 hour, 3 hour, 5 hour, 7 hour, 9 hour, 12 hour, and 24 hour. And if a customer does a 10 hour booking, it will use a 12 hour booking rate. Hence, JSON format is suitable and avoids using another table to store pricing rate, it is also scalable and easy to manage as Hasta may change their unrealistic way of price calculation in future.
features	json	NULLABLE	The list of special features of the car, JSON format is used for its scalability feature, as we can add or delete features easily without modifying tables if we use the solution of applying different tables for car features only.

5. Activations

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each activation record, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the customer ID that submit this activation request. A customer can submit multiple activation records as it might be rejected.
activated_by (FK)	bigint	FOREIGN KEY, NULLABLE	Links to the staff account that approve this activation request. It is filled once staff approve it.
ic_passport	varchar(20)	NOT NULL	IC (for Malaysian) or passport number (for foreigner) of the customer. It is not null as customers must fill this to submit an activation request. VARCHAR 20 is used because it is sufficient to occupy all IC and passport numbers in world.
nationality	varchar(50)	NOT NULL	Nationality of customer. It is not null as customers must fill this to submit an activation request. It uses varchar 2 as we are using two-letter codes (e.g. US, MY)
utm_id	varchar(20)	NOT NULL	UTM student card or staff card number, it must be unique as one student/staff only can have one account, and varchar 20 is used as it's sufficient for all UTM ID (e.g. A24CS0001) It is not null as customers must fill this to submit an activation.

contact	varchar(20)	NOT NULL	Phone contact number of customer, using varchar 20 because it is enough for all telephone numbers. It is not null as customers must fill this to submit an activation request.
emergency_ contact	varchar(20)	NOT NULL	Emergency phone contact number of customer, using varchar 20 because it is enough for all telephone numbers. It is not null as customers must fill this to submit an activation.
license_no	varchar(20)	NOT NULL	The car license number of customers as their eligibility to book a car, use varchar 20 as it is enough for all the licences. It is not null as customers must fill this to submit an activation.
license_expiry	date	NOT NULL	The expiry date of car license. It is not null as customers must fill this to submit an activation.
college	varchar(30)	NOT NULL	The residential college of students or staff in UTM. For students/staff that live outside, their college will be “NON APPLICABLE”, else it will be UTM college such as KTDI. Varchar 30 is used as it is enough for all college names. It is not null as customers must fill this to submit an activation.

faculty	varchar(30)	NOT NULL	The faculty of the customer such as Faculty of Computing, Faculty of Civil Engineering, etc. Varchar 30 is used as it is enough for all faculty names. It is not null as customers must fill this to submit an activation.
address	varchar(255)	NOT NULL	The home address of the customer, varchar 255 is enough to occupy most of the address. It is not null as customers must fill this to submit an activation.
status	varchar(20)	NOT NULL, DEFAULT 'Pending'	The status of activation record, by default is Pending, if approved turn Approved, if rejected turn Rejected. Varchar 20 is enough for all status.
reject_reason	varchar(255)	NULLABLE	The reason given by staff to reject the activation request. Varchar 255 is enough for text.

6. Vouchers

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each voucher, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the customer that uses this voucher to make a booking payment.
issued_by (FK)	bigint	FOREIGN KEY, NULLABLE	Links to the staff that assign the voucher manually, if the voucher is auto created by system like NEWUSER10 created for new users, and LOYALTY20 created for customer that claim 3 stamps, the issued_by column is null.
type	varchar(20)	NOT NULL	The type of voucher such as BOOKING10, NEWUSER10, and LOYALTY20, each voucher has their different description and different rules of discount. Varchar 20 is enough for all the types needed for a voucher.
expiry_date	date	NOT NULL	The expiry date of the voucher, exceeding expiry date will change the status of Unused voucher to Expired.
status	varchar(20)	NOT NULL, DEFAULT 'Unused'	The status of the vouchers, Unused for voucher that has not been used, Used for voucher that has been used, Expired for voucher that exceeds the expiry date. Varchar 20 is enough for all status needed for a voucher.

7. Deliveries

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each delivery location, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
location	varchar(50)	NOT NULL	The delivery locations provided by Hasta for pickup and return locations, varchar 50 is used as it is sufficient for all the locations needed,
fee	decimal(6,2)	NOT NULL	The delivery fees to deliver cars to pickup location and return location.

8. Rentals

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each rental record, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the customer account that initiates this booking.
car_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the car that is involved in this booking.
voucher_id (FK)	bigint	FOREIGN KEY, NULLABLE	Links to the voucher used in this booking, can be null if customers do not use any voucher during payment.
managed_by (FK)	bigint	FOREIGN KEY, NULLABLE	Links to the staff that manage this booking, the column become filled once any staff approve or reject booking.
pickup_date	date	NOT NULL	The selected date of car pickup by the customer.
pickup_time	time	NOT NULL	The selected time of car pickup by the customer.
return_date	date	NOT NULL	The selected date of car return by the customer.
return_time	time	NOT NULL	The selected time of car return by the customer.

pickup_location (FK)	bigint	FOREIGN KEY, NOT NULL	The selected location of car pickup by the customer. Links to the delivery location to obtain the delivery fee.
return_location (FK)	bigint	FOREIGN KEY, NOT NULL	The selected location of car return by the customer. Links to the delivery location to obtain the delivery fee.
disapprove_ reason	varchar(255)	NULLABLE	The reason to reject booking submitted by staff, this is filled if the status of booking becomes Rejected, so the customer can see the disapproval reason, else it will remain as null. Varchar 255 is enough for all reasons, and limits extremely long reasons given by staff.
feedback	varchar(255)	NULLABLE	The feedback submitted by customer after car return, it is filled if customer submitted any feedback. Varchar 255 is enough and limits extremely long feedback by customers.

9. Payments

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each transaction record, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the customer account that make this transaction record.
rental_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the rental record where this payment record belongs to.
amount	decimal(8,2)	NOT NULL	The total amount of payment of this booking, including delivery fees, deposit, full payment, and deducted by voucher discount.
refund_type	varchar(20)	NULLABLE	The refund type of the rental record, it can be Cancellation Refund to refund full if customer cancels booking before 24 hours, can be Rejection Refund to refund full if staff reject booking, can be Deposit Refund, which is RM50 after customer returns the car. Varchar 20 is enough for all types needed for refund.
refund_owner	varchar(50)	NULLABLE	The full name of the refund bank account owner, which is submitted by the customer. Varchar 50 is enough for all full names typically. It is filled once car is returned, as customers fill in return form.

refund_bank	varchar(20)	NULLABLE	The bank name of the account to be refunded. Varchar 20 is enough for all bank names. It is filled once car is returned, as customers fill in return form.
refund_account	varchar(20)	NULLABLE	The bank account number of the account to be refunded. Varchar 20 is enough for all account number. It is filled once car is returned, as customers fill in return form.
refund_amount	decimal(6,2)	NULLABLE	The total amount needs to be refunded for the customer, if the type is Cancellation Refund or Rejection, full refund will be given, else if the type is Deposit Refund, the refund will be RM50. This attribute is filled once the car is returned.

10. Penalty

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each penalty record, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	NOT NULL	Links to the customer account that is involved in this penalty record.
rental_id (FK)	bigint	NOT NULL	Links to the rental ID that causes this penalty.
issued_by (FK)	bigint	NULLABLE	Links to the staff account that manages this penalty, either approve or reject payment.
amount	decimal(6,2)	NOT NULL	The total amount of the penalty needs to be paid by the customer.

1.6 Summary

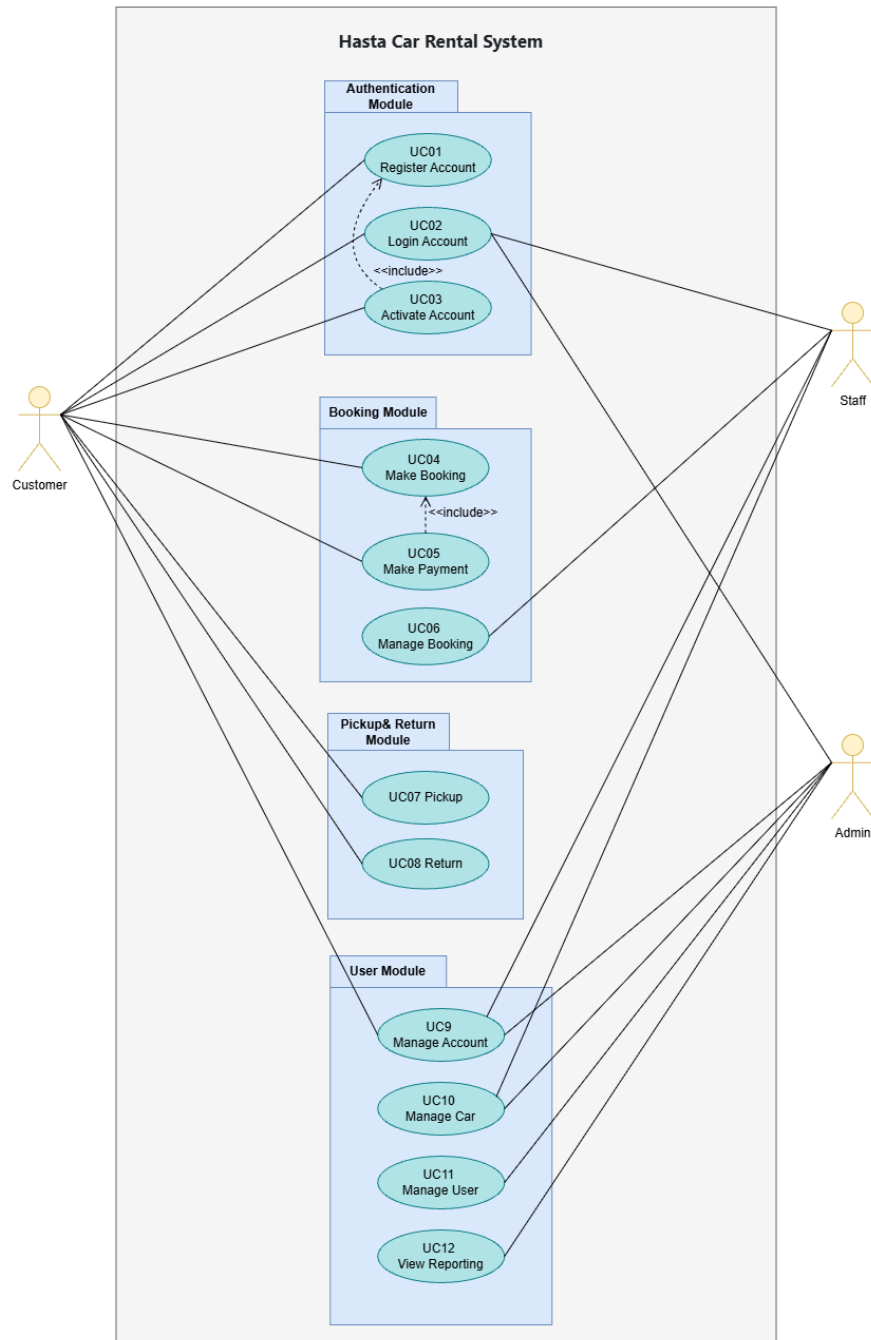
To briefly conclude this report, it is important to note that the focus of the second phase is to design the database conceptual design for the Hasta Car Rental System by transforming the requirements gathered in the first phase into a data model. In order to demonstrate the interactions between the customers, staff, admins, and the Hasta Car Rental System with regard to the key processes, a TO-BE Data Flow Diagram has been created with the following data flow processes: account activation, booking, payment, pickup and return submission, refunds, penalties, cars, and reports.

In order to achieve the data consistency and the proper functioning of the Hasta Car Rental System, a number of suggested business rules were outlined to manage the key constraints with regard to the activation blacklist limits, availability filtering for cars, bookings validation, voucher issuance, late return penalties, and stamp eligibility for the loyalty program. Moreover, the data needs for the Hasta Car Rental System were also determined with the aim to explain the data queries for the Hasta Car Rental System.

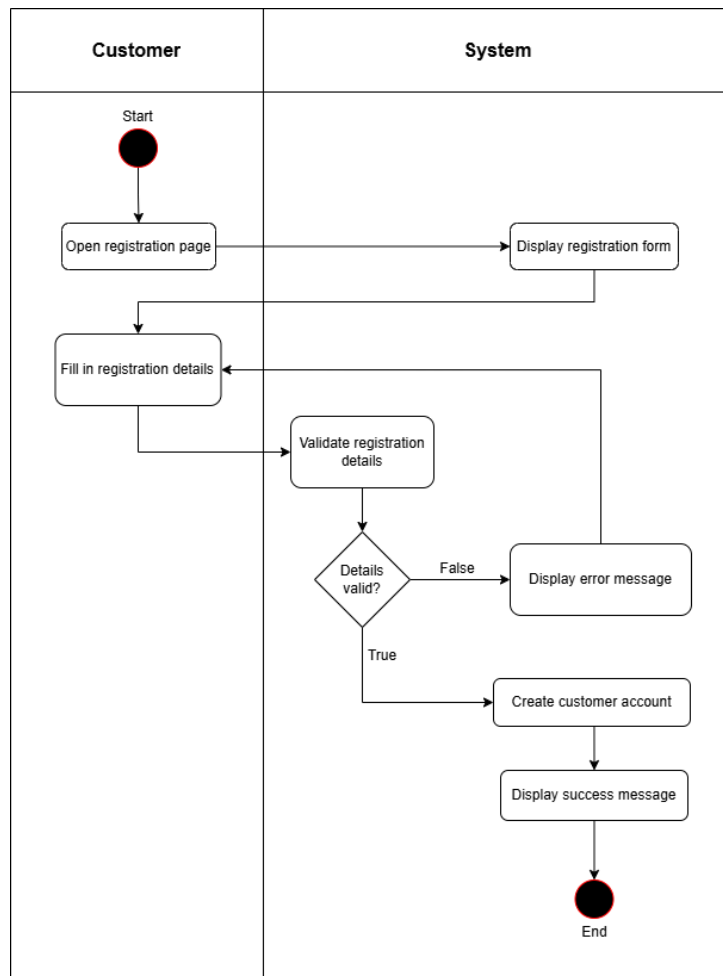
Lastly, the database structure was complete by using the Conceptual ERDs which had been based on modules, a Global ERD, and an Enhanced ERD (EERD), which indicates specialization for the Customers, Staff, and Admin. Additionally, a data dictionary had been generated which provided an outline of all the data on which the database would be implemented in the next phase.

APPENDIX

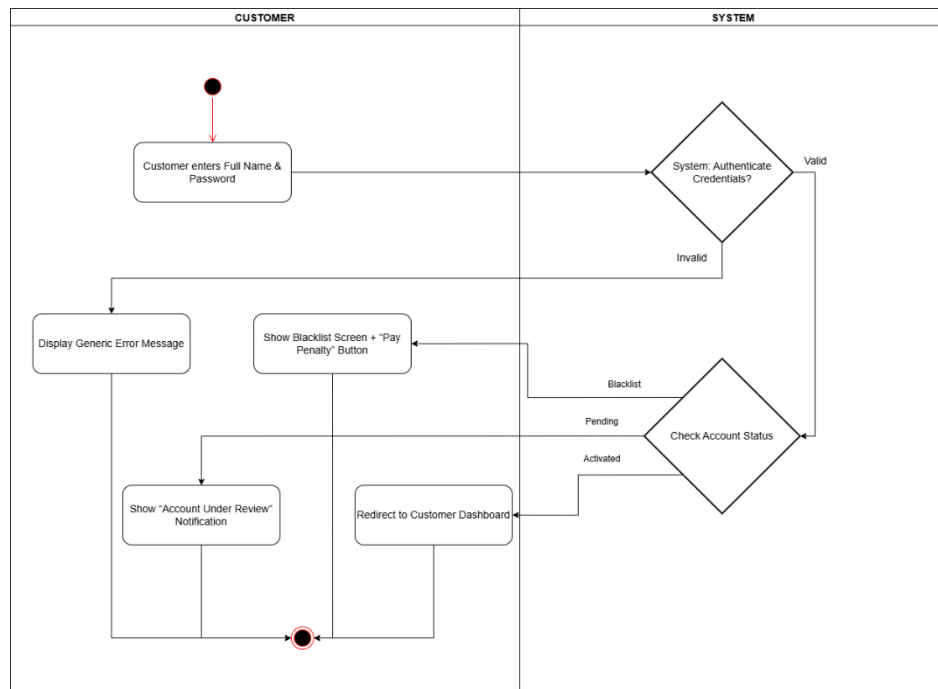
1) Use Case Diagram



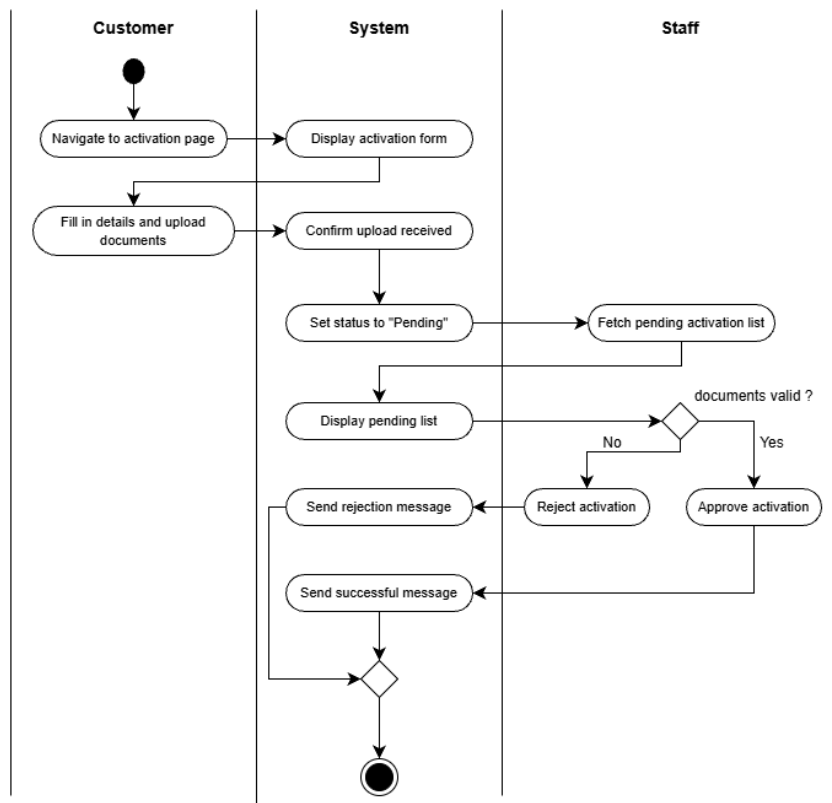
2) Activity Diagram of UC01 Register Account



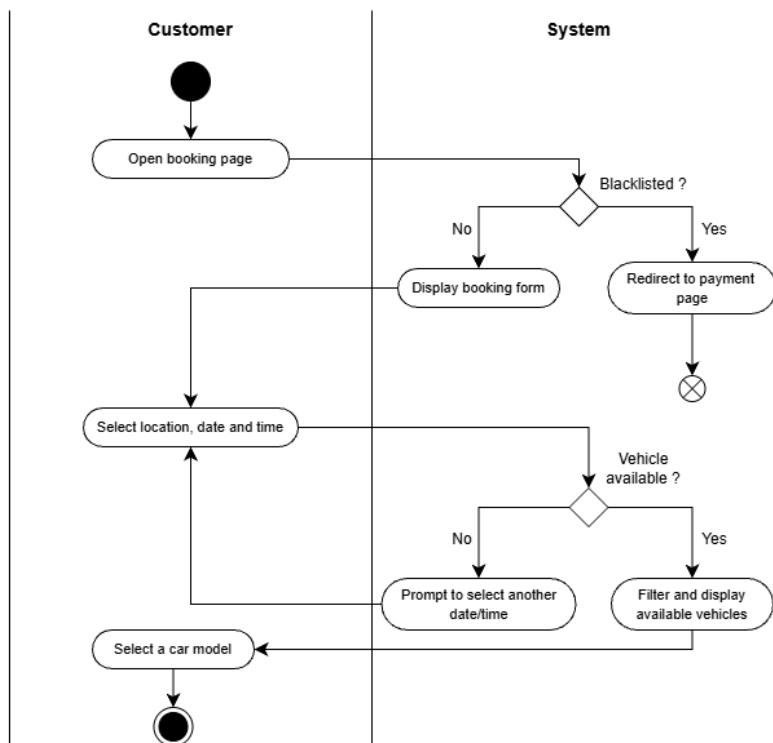
3) Activity Diagram of UC02 Login Account



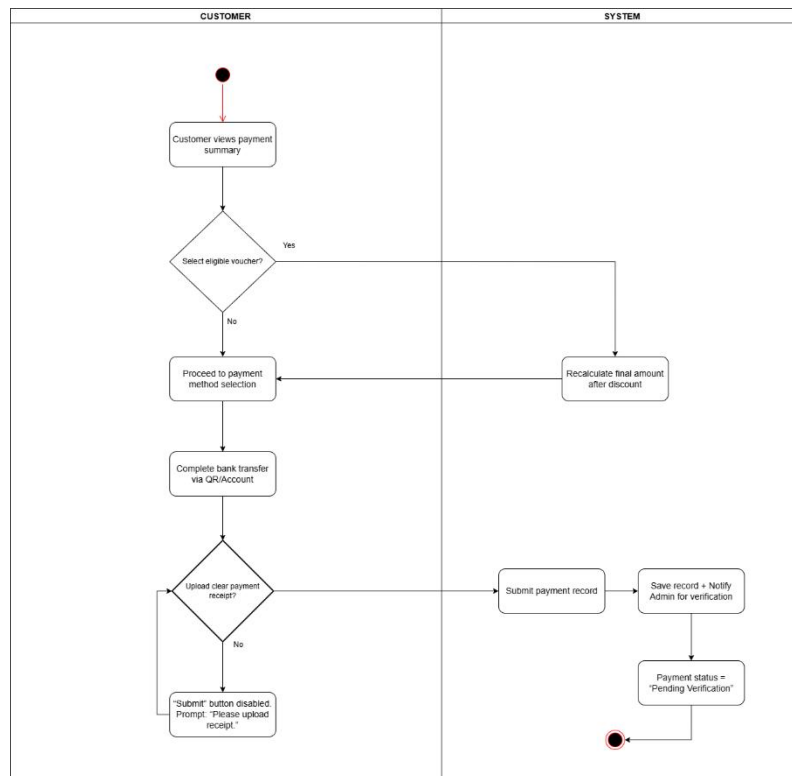
4) Activity Diagram of UC03 Activate Account



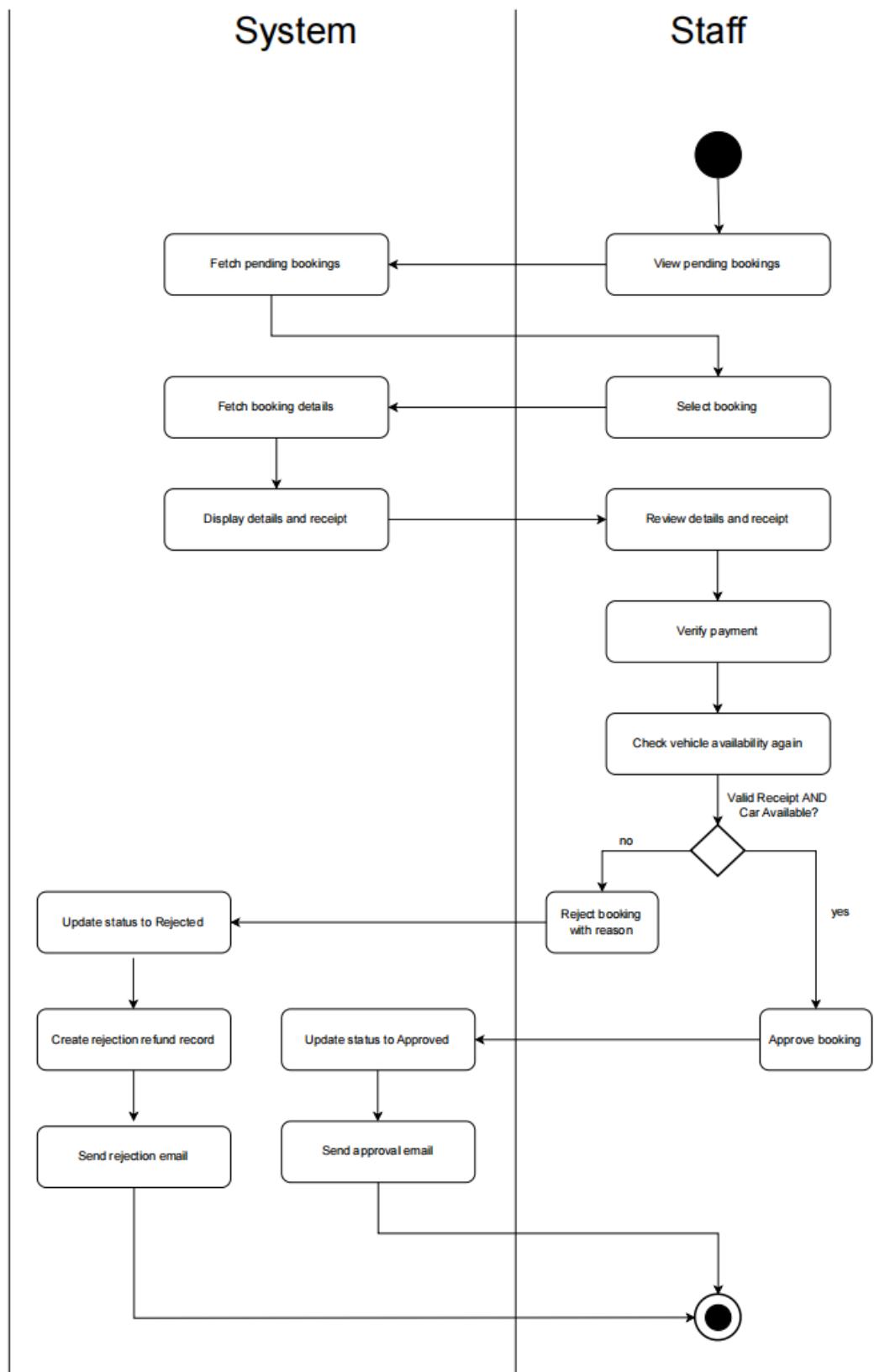
5) Activity Diagram of UC04 Make Booking



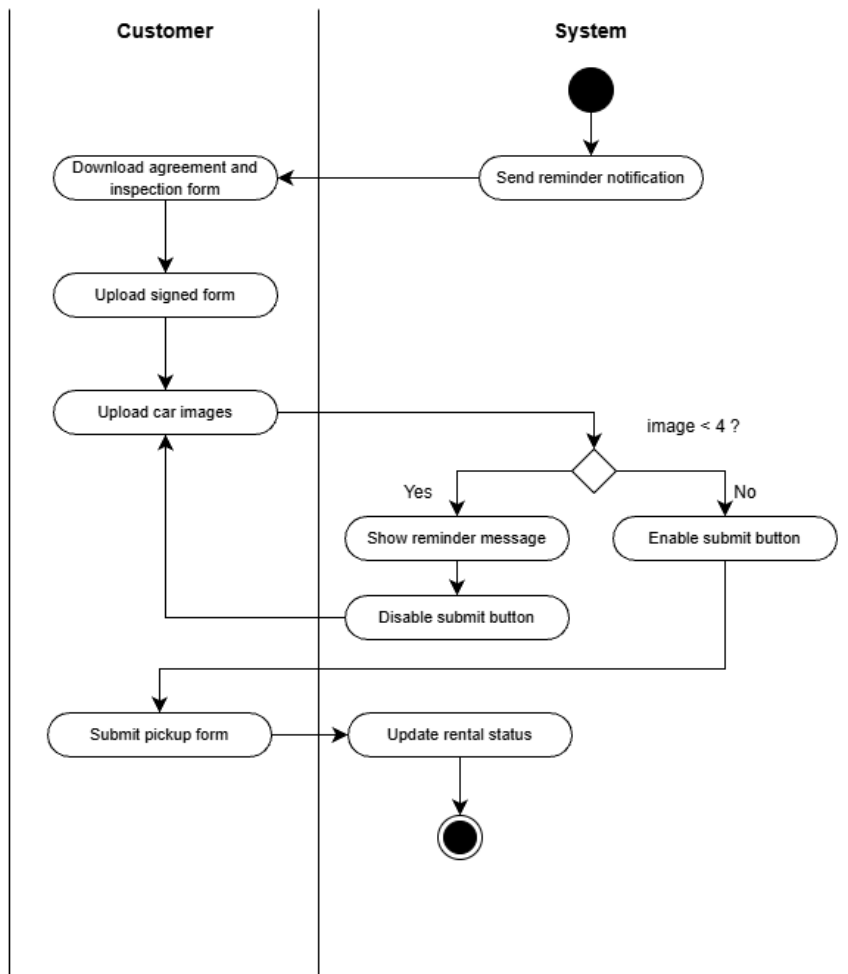
6) Activity Diagram of UC05 Make Payment



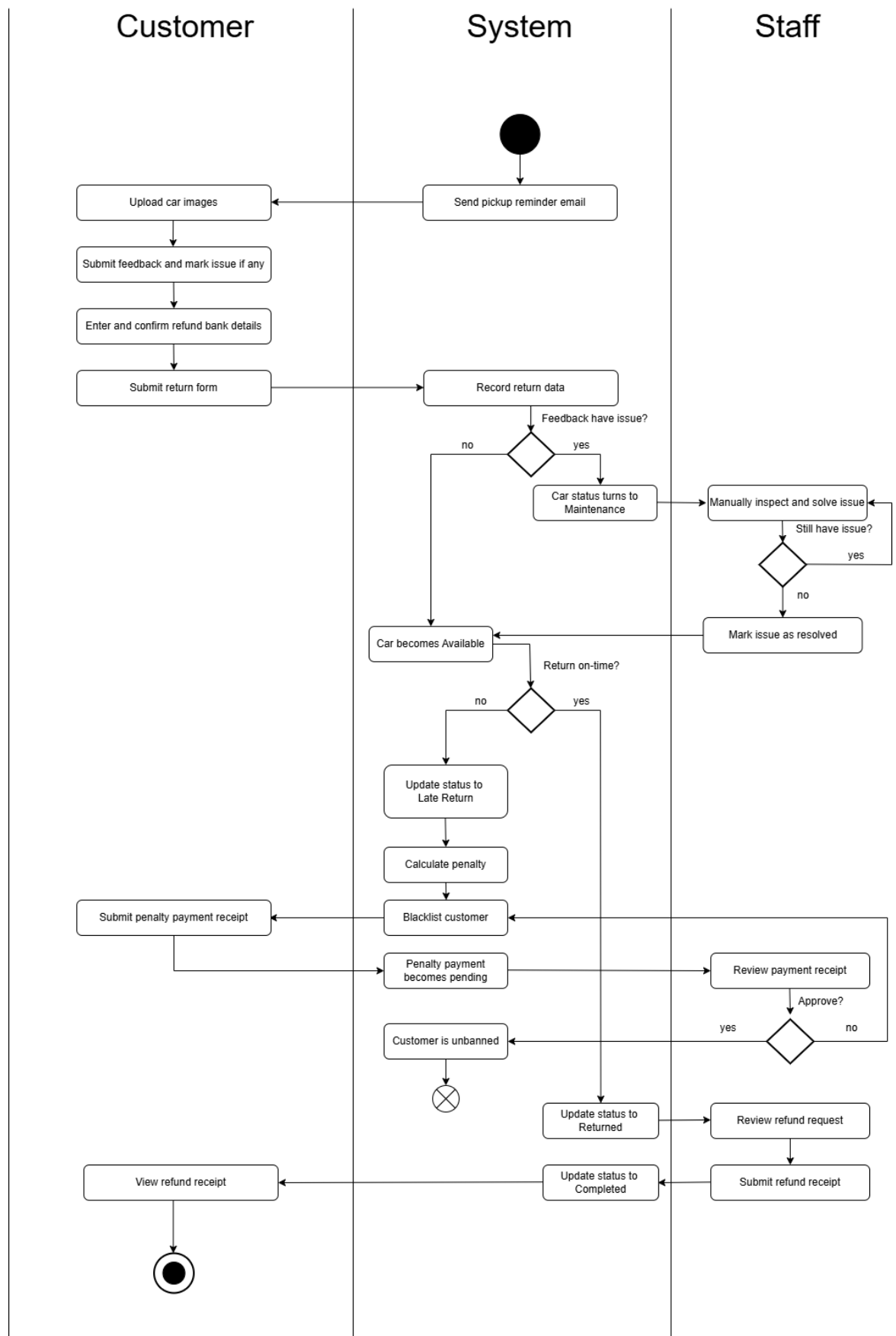
7) Activity Diagram of UC06 Manage Booking



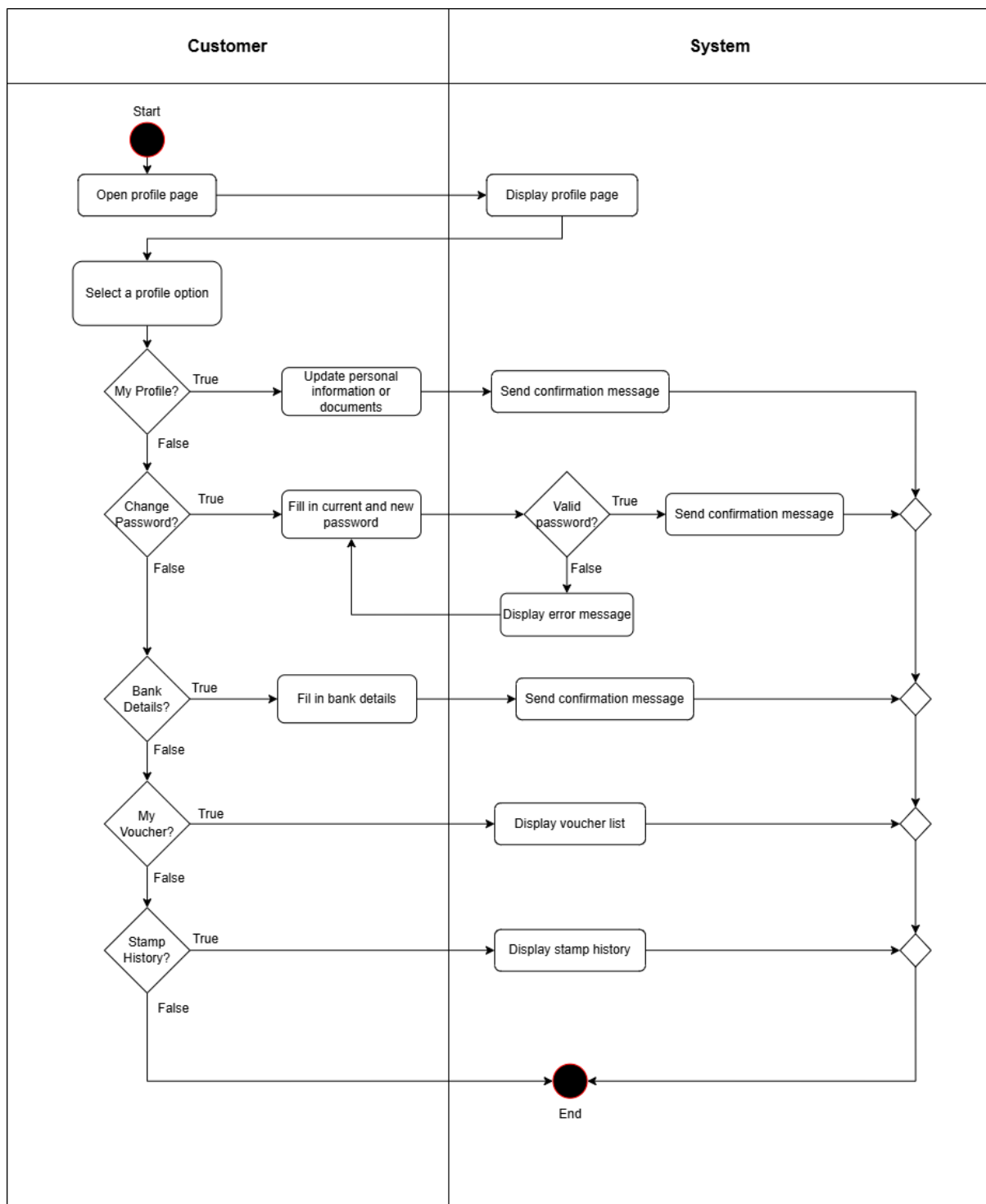
8) Activity Diagram of UC07 Pickup



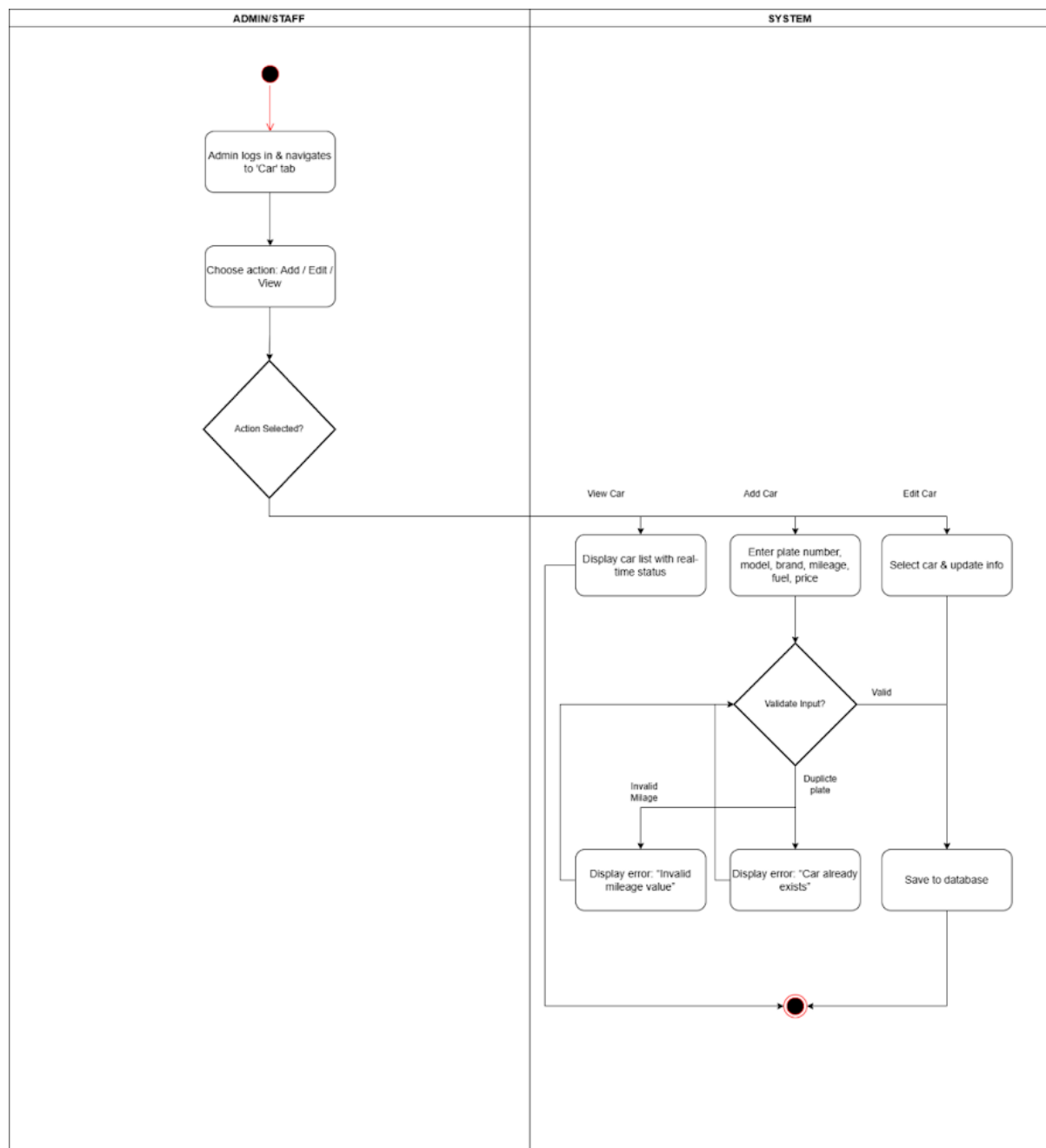
9) Activity Diagram of UC08 Return



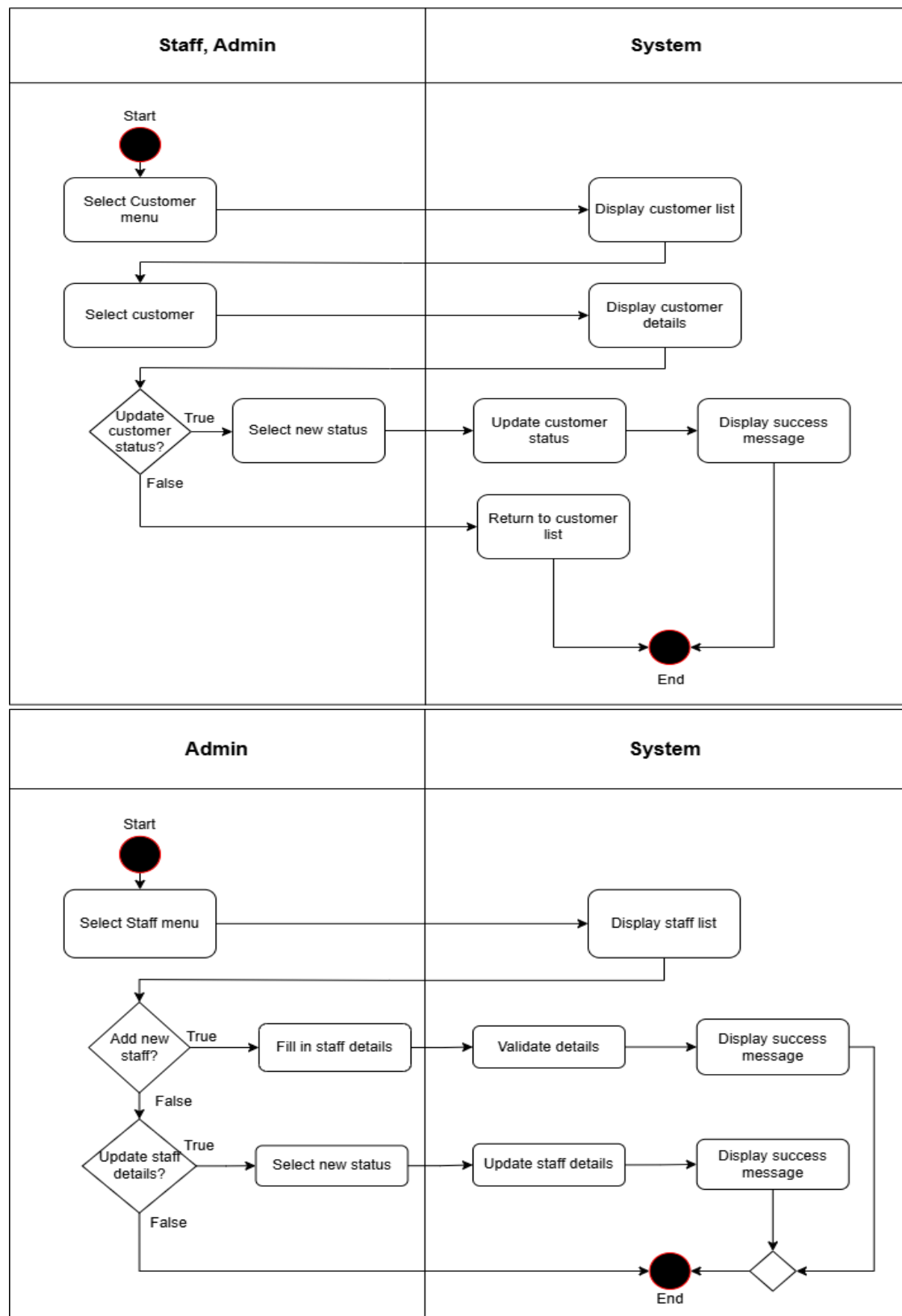
10) Activity Diagram of UC09 Manage Account



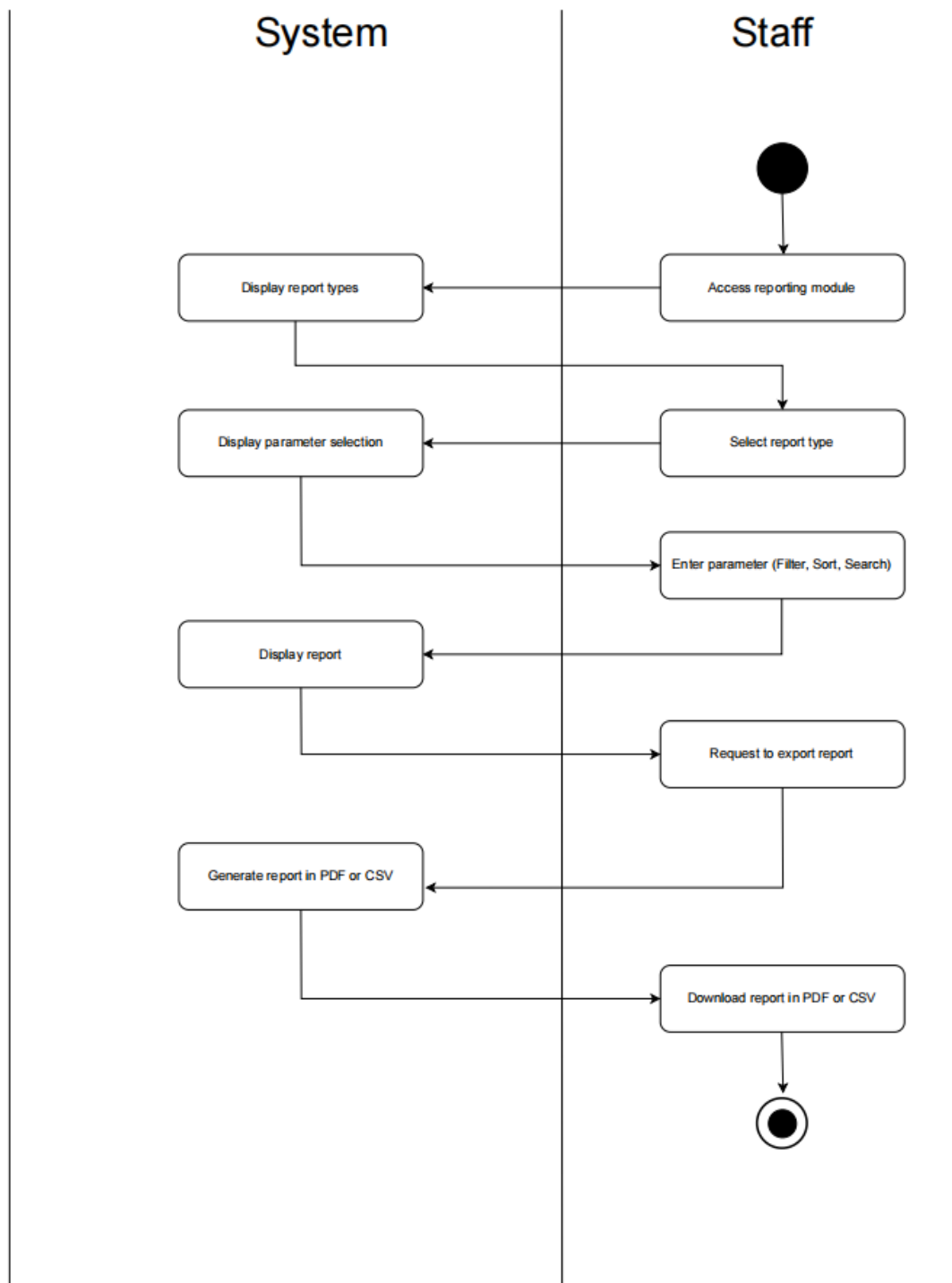
11) Activity Diagram of UC10 Manage Car



12) Activity Diagram of UC11 Manage User



13) Activity Diagram of UC12 View Reporting



Meeting Log #1

Date: 10 December 2025

Time: 9:00 p.m.

Member Involved:

1. Chun Yao Ting
2. Evelyn Ang
3. Mirza As-Siddiq bin Tohari
4. Muhammad Aliff bin Helmi

Objective:

To identify phase 2 database design deliverables and divide the tasks among members.

Discussion Summary:

In this meeting, we discussed the TO-BE system workflow and agreed on the major processes that would be included in the Data Flow Diagram (DFD). We also identified the TO-BE data requirements, TO-BE transaction requirements, and proposed business rules based on the agreed workflow of the system. After that, we also discussed how to translate the identified requirements into database artefacts, creating the Conceptual ERD, Enhanced ERD (EERD), and Data Dictionary.

	Task Distributed	Person-In-Charge
1.	Data Flow Diagram	Mirza As-Siddiq bin Tohari
2.	TO-BE Data Requirements	Chun Yao Ting
3.	TO-BE Transaction Requirements	Evelyn Ang
4.	TO-BE Business Rules	Muhammad Aliff bin Helmi
5.	Conceptual ERD	- Chun Yao Ting - Evelyn Ang
6.	Enhanced ERD	- Mirza As-Siddiq bin Tohari - Muhammad Aliff bin Helmi
7.	Data Dictionary	All members

